QGenomix — Complete Project Plan

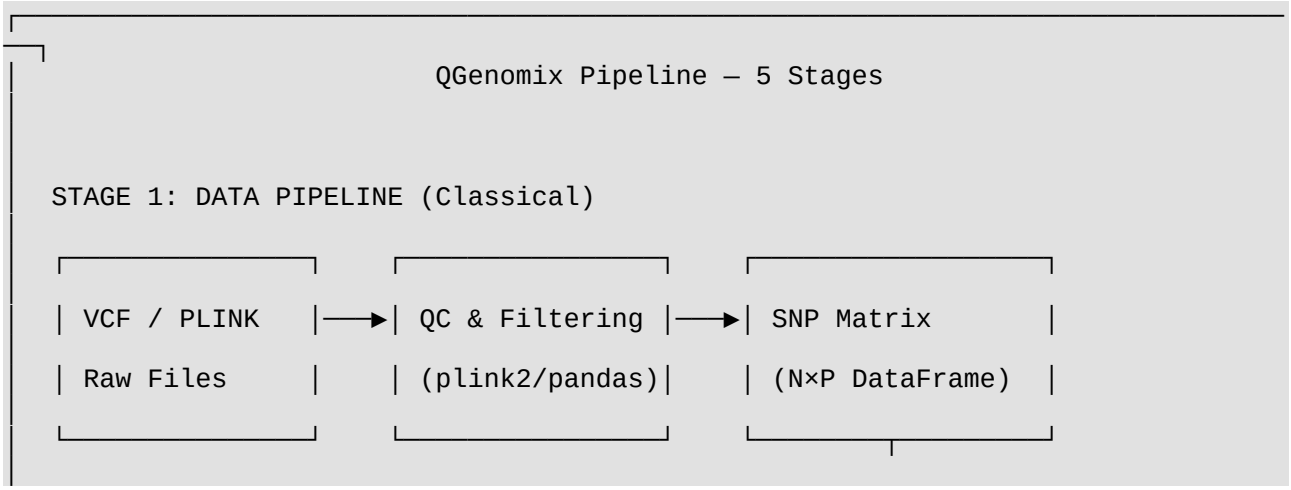
Field	Value
Project	QGenomix: A Quantum-Classical Hybrid Framework for Polygenic Disease Risk Prediction
Generated	2026-05-25
Agent Model	Claude Opus 4.6 (Thinking)
Estimated Word Count	~19,000
Document Version	2.0 (Fault-corrected)

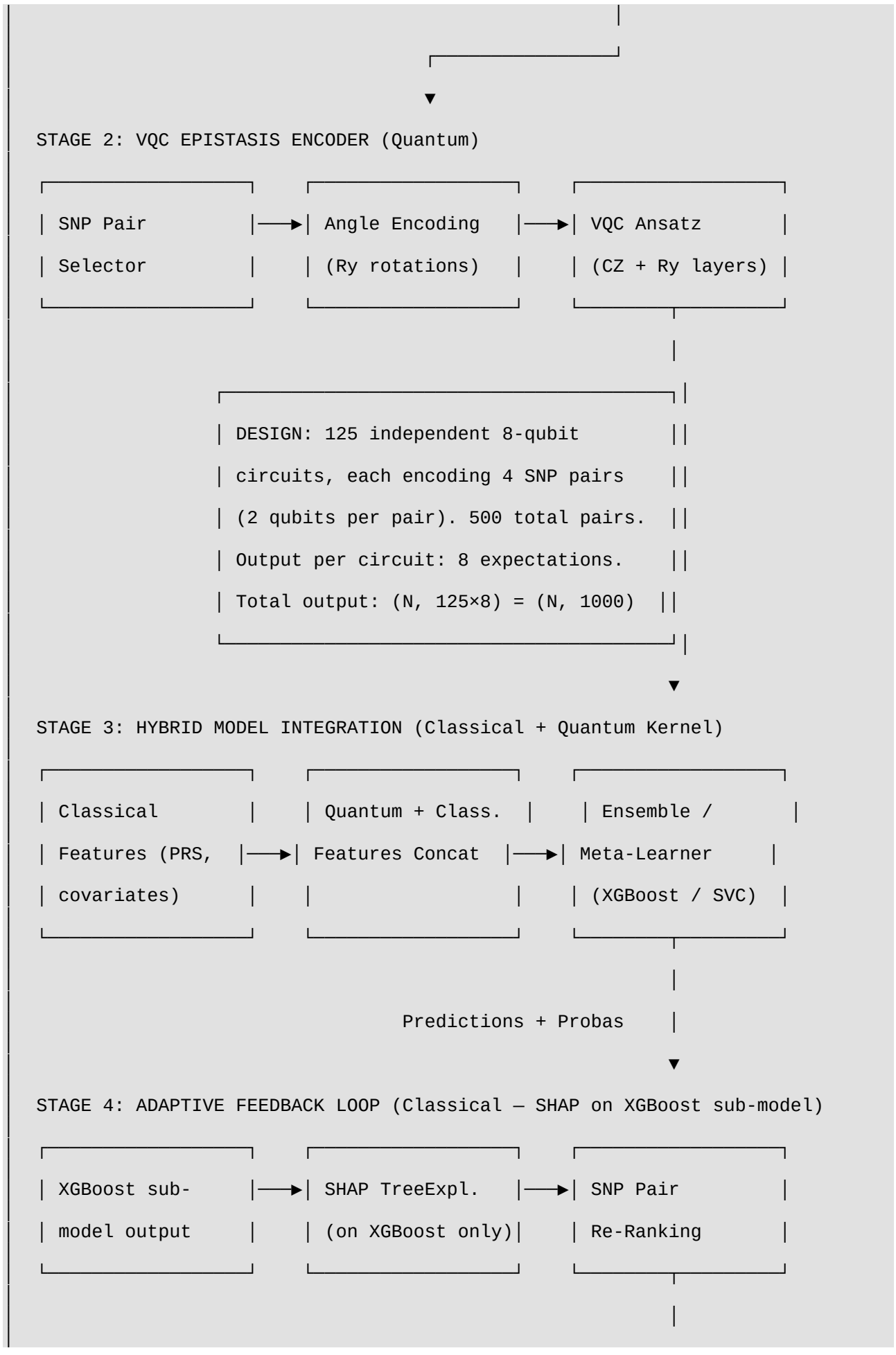
Table of Contents

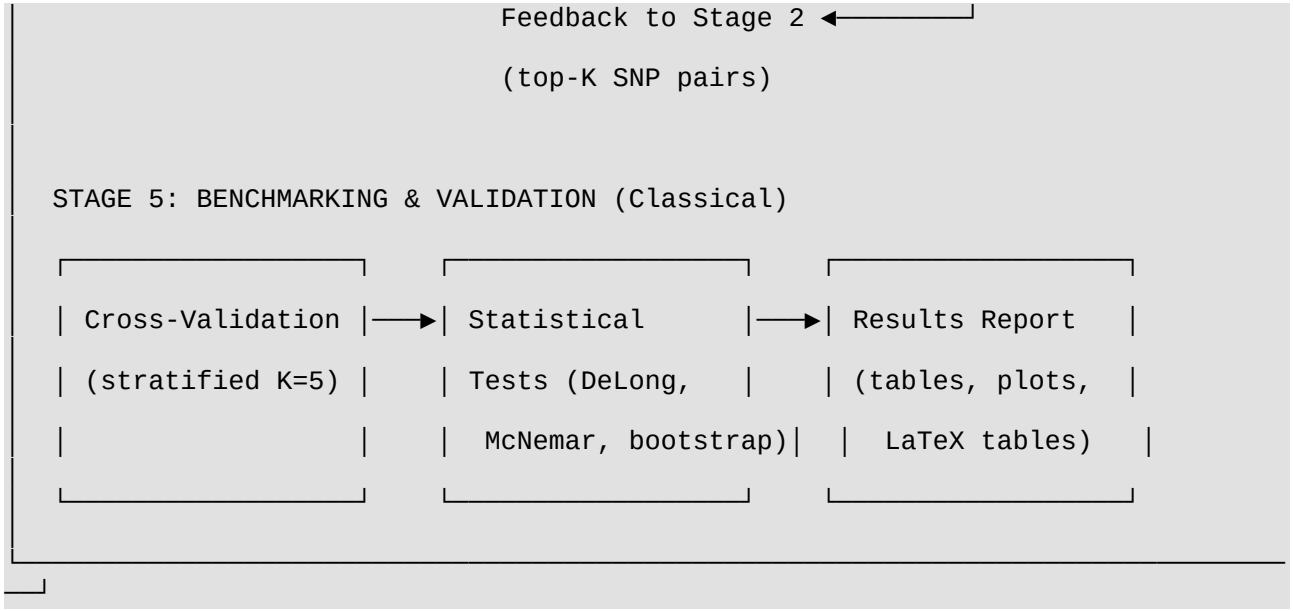
- Section A — Project Architecture
- Section B — Complete File Tree Structure
- Section C — Chapter / Phase / Task / Sub-Task Breakdown
- Section D — 3-Person Team Assignment
- Section E — Tools Reference Guide
- Section F — Standard Production Procedures
- Section G — Risk Register & Mitigation Plan
- Section H — Glossary

Section A — Project Architecture

A.1 — Component Diagram




STAGE 2: VQC EPISTASIS ENCODER (Quantum)
SNP Pair Selector
Angle Encoding (Ry rotations)
VQC Ansatz (CZ + Ry layers)
DESIGN: 125 independent 8-qubit circuits, each encoding 4 SNP pairs (2 qubits per pair). 500 total pairs. Output per circuit: 8 expectations. Total output: (N, 125×8) = (N, 1000)
STAGE 3: HYBRID MODEL INTEGRATION (Classical + Quantum Kernel)
Classical Features (PRS, covariates)
Quantum + Class. Features Concat
Ensemble / Meta-Learner (XGBoost / SVC)
Predictions + Probas
STAGE 4: ADAPTIVE FEEDBACK LOOP (Classical — SHAP on XGBoost sub-model)
XGBoost sub-model output
SHAP TreeExpl. (on XGBoost only)
SNP Pair Re-Ranking



A.2 — Module Map

Module Path	Responsibility	Inputs	Outputs
`src/data/__init__.py`	Package init for data module	—	—
`src/data/download.py`	Download 1000 Genomes PLINK files & phenotype data	URLs, config	Raw `.pgen/.pvar/.psam` + phenotype CSV
`src/data/qc.py`	Quality control: MAF filtering, HWE, missingness, LD pruning via plink2	Raw PLINK files, QC thresholds	Filtered PLINK files
`src/data/preprocess.py`	Convert filtered PLINK to SNP matrix via `plink2 --export A` + Pandas, impute missing, standardize	Filtered PLINK files	`np.ndarray` shape `(N, P)`, phenotype `np.ndarray` shape `(N,)`
`src/data/epistasis_pairs.py`	Select candidate SNP pairs via GWAS p-value product ranking and LD-aware pruning	SNP matrix, GWAS summary stats	List of `(snp_i, snp_j)` tuples
`src/data/simulate_phenotype.py`	Generate epistatic phenotype labels using injected interaction effects (NOT from PRS betas)	SNP matrix, interaction config	Phenotype vector `(N,)`
`src/data/dataset.py`	PyTorch `Dataset` wrapper for SNP data + phenotypes; supports `mode='pairs'` and	SNP matrix, labels, pair indices	`torch.utils.data.Dataset`

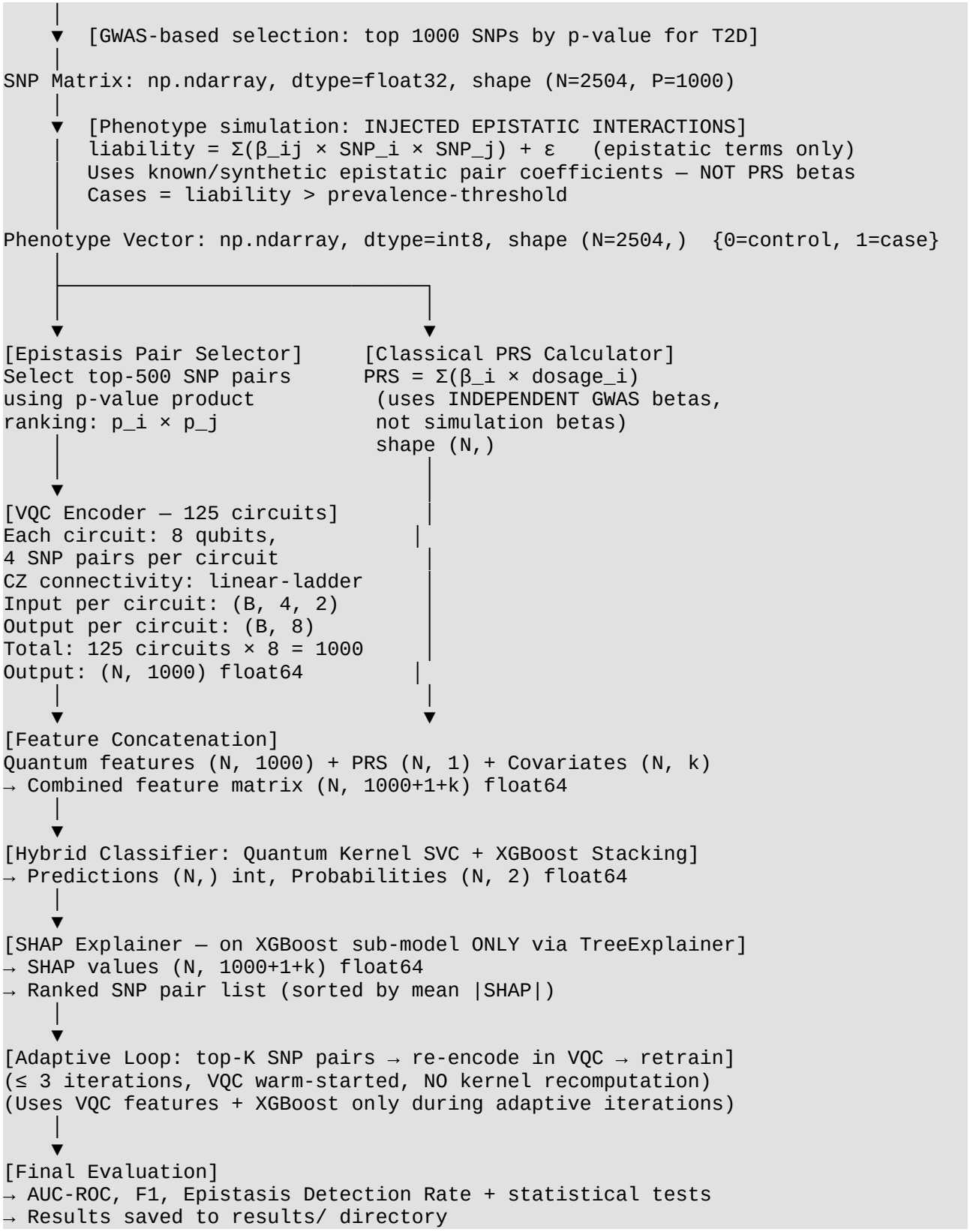
Module Path	Responsibility	Inputs	Outputs
	`mode='matrix'`		
`src/quantum/__init__.py`	Package init for quantum module	—	—
`src/quantum/encoder.py`	`encode_snp_pair(snp_a, snp_b) -> circuit`: angle-encode 2 SNP values into Ry rotations on 2 qubits	Two float values `[0, π]`	Quantum state on 2 qubits
`src/quantum/vqc.py`	`VQCEpistasis(n_qubits=8, n_layers, n_pairs_per_circuit=4)`: 8-qubit VQC packing 4 SNP pairs per circuit. CZ connectivity: **linear-ladder** with configurable alternative (circular).	SNP pair batch `(B, 4, 2)`	Expectation values `(B, 8)` per circuit
`src/quantum/kernel.py`	`QuantumKernel(n_qubits)`: compute quantum kernel matrix K[i,j] via fidelity. Input is VQC feature vector per sample.	Feature vectors `(N, d)` where `d = n_qubits`	Kernel matrix `(N, N)` float64
`src/quantum/train_vqc.py`	Training loop for VQC using parameter-shift rule via PennyLane + PyTorch	Dataset, VQC config	Trained VQC weights `.pt` file
`src/classical/__init__.py`	Package init for classical module	—	—
`src/classical/prs.py`	Compute standard PRS (linear weighted sum of effect alleles). **Note:** uses independent GWAS weights, NOT the same betas used for phenotype simulation.	SNP matrix, GWAS weights	PRS vector `(N,)`
`src/classical/baselines.py`	Train classical baselines: Logistic Regression, Random Forest, XGBoost, SVM	SNP matrix, labels, config	Trained sklearn/xgb models
`src/classical/metrics.py`	Compute AUC-ROC, F1, epistasis detection rate, confusion matrix	Predictions, true labels	Dict of metric values

Module Path	Responsibility	Inputs	Outputs
`src/hybrid/__init__.py`	Package init for hybrid module	—	—
`src/hybrid/feature_concat.py`	Concatenate quantum features (VQC outputs), PRS scores, and classical covariates into a single matrix	VQC output `(N, 1000)`, PRS `(N, 1)`, covariates `(N, k)`	Combined `(N, 1000+1+k)`
`src/hybrid/model.py`	`HybridClassifier`: wraps quantum kernel SVC + XGBoost ensemble via stacking	Combined features, kernel matrix	Predictions, probabilities
`src/hybrid/train.py`	End-to-end hybrid model training pipeline	Config, data paths	Trained hybrid model artifacts
`src/explainability/__init__.py`	Package init for explainability module	—	—
`src/explainability/shap_analysis.py`	Compute SHAP values **on the XGBoost sub-model only** (via TreeExplainer), rank SNP pairs. Does NOT use KernelExplainer on full hybrid.	Trained XGBoost, feature matrix	SHAP values `(N, P)`, ranked pair list
`src/explainability/adaptive_loop.py`	Re-rank SNP pairs using SHAP, feed top-K back to VQC, retrain (≤3 iterations). VQC weights **warm-started** from previous iteration. Recomputes VQC features but uses XGBoost+VQC-features only (no kernel recomputation during adaptive loop).	SHAP rankings, VQC, data	Updated VQC weights, iteration log
`src/explainability/visualize.py`	SHAP summary plots, waterfall plots, interaction plots	SHAP values, feature names	PNG/SVG plot files
`src/benchmark/__init__.py`	Package init for benchmark module	—	—
`src/benchmark/evaluate.py`	Cross-validation evaluation: stratified K-fold, per-fold metrics. Includes explicit kernel matrix subsetting: `K_train =`	Model, data, CV config	Per-fold metrics DataFrame

Module Path	Responsibility	Inputs	Outputs
	`K[np.ix_(train_idx, train_idx)]`, `K_test = K[np.ix_(test_idx, train_idx)]`		
`src/benchmark/statistical_tests.py`	DeLong test for AUC comparison, McNemar test, bootstrap CIs	Two sets of predictions	p-values, confidence intervals
`src/benchmark/report.py`	Generate LaTeX tables, comparison plots, final results summary	All metric DataFrames	`results/` output files
`src/utils/__init__.py`	Package init for utils module	—	—
`src/utils/config.py`	Load/validate YAML config files	Config file path	Dict config
`src/utils/logging.py`	Structured logging setup (console + file)	—	Logger instance
`src/utils/reproducibility.py`	Set all random seeds (Python, NumPy, PyTorch, PennyLane)	Seed int	—
`src/utils/io.py`	Save/load models, arrays, results to/from disk	Object, path	—
`scripts/run_pipeline.py`	CLI entry point: run full pipeline or individual stages	CLI args	—
`scripts/download_data.sh`	Shell script to download 1000 Genomes + phenotype data	—	Raw data files
`scripts/run_qc.sh`	Shell script to run plink2 QC steps	Raw data path	Filtered data
`scripts/hpc_job.slurm`	SLURM job script template for HPC	—	—

A.3 — Data Flow

```
Raw PLINK files (1000 Genomes Phase 3 — available as genome-wide merged file)
(~2504 samples × ~80M variants)
    │
    ▼  [plink2 QC: --maf 0.01 --hwe 1e-6 --geno 0.05 --mind 0.1]
    │
Filtered PLINK files (~2504 samples × ~5M variants)
    │
    ▼  [LD pruning: --indep-pairwise 50 5 0.2 → ~500K tag SNPs]
```



A.4 — Stateless vs. Stateful Components

Component	Type	Notes
`data/download.py`	Stateless	Idempotent download — can re-run safely

Component	Type	Notes
`data/qc.py`	Stateless	Deterministic given same inputs + thresholds
`data/preprocess.py`	Stateless	Deterministic with fixed seed
`data/simulate_phenotype.py`	Stateless	Deterministic with fixed seed + interaction config
`data/epistasis_pairs.py`	Stateless	Deterministic given GWAS stats
`quantum/vqc.py`	**Stateful**	Trainable parameters persist across epochs
`quantum/kernel.py`	Stateless	Kernel matrix computed fresh from feature map
`quantum/train_vqc.py`	**Stateful**	Writes trained weights; depends on optimizer state
`classical/baselines.py`	**Stateful**	Trained model objects persist
`hybrid/model.py`	**Stateful**	Combines trained sub-models
`explainability/adaptive_loop.py`	**Stateful**	Modifies VQC weights across iterations (warm-started)
`benchmark/evaluate.py`	Stateless	Runs on frozen models

A.5 — Integration Points (Quantum ↔ Classical Boundary)

Boundary	From	To	Tensor Shape	dtype	Device
SNP → VQC	`preprocess.py`	`vqc.py`	`(batch, 4, 2)` per circuit — 4 pairs packed into one 8-qubit circuit	`float32`	CPU → `default.qubit` simulator
VQC → Classical	`vqc.py`	`feature_concat.py`	`(N, 1000)` — 125 circuits × 8 expectations concatenated	`float64`	Simulator → CPU
Features → Kernel	VQC output (reduced)	`kernel.py`	`(N, 8)` — PCA-reduced or single-circuit summary for kernel	`float64`	CPU → `default.qubit` → CPU
Kernel → SVC	`kernel.py`	`model.py`	`(N, N)`	`float64`	CPU
SHAP → VQC	`adaptive_l`	`train_vqc.`	`List[(int,`	Python list	CPU

Boundary	From	To	Tensor Shape	dtype	Device
	`oop.py`	`py`	`int)]` top-K pairs		

Section B — Complete File Tree Structure

B.1 — Final Complete File Tree

```
qgenomix/
├── .github/
│   └── workflows/
│       ├── ci.yml                    # GitHub Actions: lint + test on push/PR
│       └── docs.yml                  # GitHub Actions: build & deploy docs
├── .gitignore                        # Ignore data/, results/, *.pyc, .env,
etc.
├── .env.example                      # Template for environment variables
├── .pre-commit-config.yaml           # Pre-commit hooks config (black, flake8,
mypy)
├── LICENSE                           # MIT License
├── Makefile                          # Build automation (install, data, train,
test, etc.)
├── README.md                         # Project overview, setup, usage, results
summary
├── pyproject.toml                    # Project metadata, build config, tool
settings
├── requirements.txt                  # Production dependencies (pinned
versions)
├── requirements-dev.txt              # Dev dependencies (pytest, black, flake8,
mypy)
├── setup.py                          # Legacy setuptools entry point
├── QGenomix_Project_Plan.md          # This document
├── configs/
│   ├── default.yaml                  # Default pipeline configuration
│   ├── vqc_8qubit.yaml               # VQC config: 8 qubits, 6 layers
│   ├── vqc_scaling.yaml              # VQC qubit-scaling experiment: 4, 6, 8,
10 qubits
│   ├── classical_baselines.yaml      # Config for all classical baseline models
│   ├── shap_config.yaml              # SHAP analysis parameters
│   └── experiment_tracking.yaml      # Experiment naming, seed, logging config
├── data/
│   ├── README.md                     # Data provenance, download instructions,
checksums
│   ├── checksums.sha256              # SHA256 checksums for all processed data
files
│   ├── raw/                          # Raw downloaded files (NEVER committed)
│   │   └── .gitkeep
│   ├── interim/                      # Intermediate QC outputs (NEVER
committed)
│   │   └── .gitkeep
│   ├── processed/                    # Final processed SNP matrices (NEVER
committed)
│   │   └── .gitkeep
│   └── external/                     # External GWAS summary stats, phenotype
mappings
│       └── .gitkeep
├── src/
│   ├── __init__.py                   # Root package init
```

```
│   ├── data/
│   │   ├── __init__.py              # Data package init
│   │   ├── download.py              # Download 1000 Genomes + phenotype data
│   │   ├── qc.py                    # Quality control via plink2
│   │   ├── preprocess.py            # SNP matrix construction, imputation,
scaling
│   │   ├── simulate_phenotype.py    # Epistatic phenotype simulation (NOT from
PRS betas)
│   │   ├── epistasis_pairs.py       # GWAS-based SNP pair candidate selection
│   │   └── dataset.py               # PyTorch Dataset wrapper (supports pairs
and matrix modes)
│   ├── quantum/
│   │   ├── __init__.py              # Quantum package init
│   │   ├── encoder.py               # SNP pair → angle encoding circuit
│   │   ├── vqc.py                   # VQC Epistasis Encoder (CZ linear-ladder
+ Ry ansatz)
│   │   ├── kernel.py                # Quantum Kernel matrix computation
│   │   └── train_vqc.py             # VQC training loop (parameter-shift rule)
│   ├── classical/
│   │   ├── __init__.py              # Classical package init
│   │   ├── prs.py                   # Polygenic Risk Score computation
│   │   ├── baselines.py             # LR, RF, XGBoost, SVM baseline training
│   │   └── metrics.py               # AUC-ROC, F1, epistasis detection rate
│   ├── hybrid/
│   │   ├── __init__.py              # Hybrid package init
│   │   ├── feature_concat.py        # Quantum + PRS + classical feature
concatenation
│   │   ├── model.py                 # HybridClassifier (quantum kernel SVC +
XGBoost)
│   │   └── train.py                 # End-to-end hybrid training pipeline
│   ├── explainability/
│   │   ├── __init__.py              # Explainability package init
│   │   ├── shap_analysis.py         # SHAP value computation (TreeExplainer on
XGBoost only)
│   │   ├── adaptive_loop.py         # Adaptive SNP re-ranking + VQC retraining
│   │   └── visualize.py             # SHAP plots (summary, waterfall,
interaction)
│   ├── benchmark/
│   │   ├── __init__.py              # Benchmark package init
│   │   ├── evaluate.py              # Stratified K-fold cross-validation
│   │   ├── statistical_tests.py     # DeLong, McNemar, bootstrap CI tests
│   │   └── report.py                # LaTeX tables, comparison plots, summary
│   └── utils/
│       ├── __init__.py              # Utils package init
│       ├── config.py                # YAML config loader + validator
│       ├── logging.py               # Structured logging setup
│       ├── reproducibility.py       # Global seed setting
│       └── io.py                    # Model / array save / load helpers
├── scripts/
│   ├── run_pipeline.py              # CLI entry point for full / partial
pipeline
│   ├── download_data.sh             # Bash script: download all raw data
│   ├── run_qc.sh                    # Bash script: run plink2 QC pipeline
│   ├── download_models.sh           # Download pre-trained models from
external storage
│   └── hpc_job.slurm                # SLURM job script template
├── notebooks/
│   ├── 01_data_exploration.ipynb    # EDA on raw genomic data
│   ├── 02_vqc_demo.ipynb            # Interactive VQC training demo
│   ├── 03_quantum_kernel_demo.ipynb  # Quantum kernel SVM demo
│   ├── 04_hybrid_pipeline.ipynb     # Full hybrid pipeline walkthrough
│   ├── 05_shap_explainability.ipynb  # SHAP analysis + adaptive loop demo
│   └── 06_final_results.ipynb       # Reproducible final results notebook
├── tests/
```

```
│   ├── __init__.py                    # Tests package init
│   ├── conftest.py                    # Shared pytest fixtures
│   ├── test_data/
│   │   ├── __init__.py
│   │   ├── test_preprocess.py         # Test SNP matrix construction
│   │   └── test_epistasis_pairs.py    # Test pair selection logic
│   ├── test_quantum/
│   │   ├── __init__.py
│   │   ├── test_encoder.py            # Test angle encoding correctness
│   │   ├── test_vqc.py                # Test VQC output shapes + gradient flow
│   │   └── test_kernel.py             # Test kernel matrix symmetry + PSD
│   ├── test_classical/
│   │   ├── __init__.py
│   │   ├── test_prs.py                # Test PRS computation
│   │   ├── test_baselines.py          # Test baseline model training
│   │   └── test_metrics.py            # Test metric calculations
│   ├── test_hybrid/
│   │   ├── __init__.py
│   │   └── test_model.py              # Test hybrid classifier end-to-end
│   └── test_explainability/
│       ├── __init__.py
│       ├── test_shap.py               # Test SHAP computation
│       └── test_adaptive_loop.py      # Test adaptive loop convergence
├── docs/
│   ├── mkdocs.yml                     # MkDocs configuration
│   ├── index.md                       # Documentation home page
│   ├── setup_guide.md                 # Installation + environment setup
│   ├── data_guide.md                  # Data acquisition + preprocessing guide
│   ├── architecture.md                # System architecture documentation
│   ├── api_reference.md               # Auto-generated API docs
│   ├── vqc_design.md                  # VQC design rationale + math
│   ├── quantum_kernel.md              # Quantum kernel theory + implementation
│   ├── experiment_log.md              # Running experiment log
│   └── assets/                        # Images, diagrams for docs
│       └── .gitkeep
└── results/
    ├── README.md                      # Results directory structure + naming
    ├── figures/                       # Generated plots and figures
    │   └── .gitkeep
    ├── tables/                        # LaTeX / CSV result tables
    │   └── .gitkeep
    ├── models/                        # Saved model checkpoints — hosted
externally (Google Drive/Zenodo)
    │   └── .gitkeep
    └── logs/                          # Training logs, experiment logs
        └── .gitkeep
```

B.2 — Files by Phase

After Phase 1 (Weeks 1–2): Data Pipeline

```
▫ .gitignore, .env.example, README.md, Makefile, pyproject.toml
▫ requirements.txt, requirements-dev.txt, setup.py
▫ configs/default.yaml
▫ data/README.md, data/checksums.sha256, data/raw/, data/interim/,
data/processed/, data/external/
▫ src/__init__.py
▫ src/data/__init__.py, download.py, qc.py, preprocess.py,
simulate_phenotype.py, epistasis_pairs.py, dataset.py
▫ src/utils/__init__.py, config.py, logging.py, reproducibility.py, io.py
▫ scripts/download_data.sh, run_qc.sh
▫ tests/conftest.py, tests/test_data/test_preprocess.py, test_epistasis_pairs.py
```

- notebooks/01_data_exploration.ipynb

After Phase 2 (Weeks 3–5): VQC Design & Training

- All Phase 1 files +
- configs/vqc_8qubit.yaml, vqc_scaling.yaml
- src/quantum/__init__.py, encoder.py, vqc.py, kernel.py, train_vqc.py
- tests/test_quantum/test_encoder.py, test_vqc.py, test_kernel.py
- notebooks/02_vqc_demo.ipynb, 03_quantum_kernel_demo.ipynb

After Phase 3 (Weeks 6–7): Classical Baselines

- All Phase 2 files +
- configs/classical_baselines.yaml
- src/classical/__init__.py, prs.py, baselines.py, metrics.py
- tests/test_classical/test_prs.py, test_baselines.py, test_metrics.py

After Phase 4 (Weeks 8–11): Hybrid Integration & Benchmarking

- All Phase 3 files +
- src/hybrid/__init__.py, feature_concat.py, model.py, train.py
- src/benchmark/__init__.py, evaluate.py, statistical_tests.py
- tests/test_hybrid/test_model.py
- notebooks/04_hybrid_pipeline.ipynb
- scripts/run_pipeline.py, hpc_job.slurm

> **Note (Fault #41 fix):** `src/benchmark/report.py` is built in Chapter 6 (weeks 14–15), not Phase 4. Phase 4 builds the evaluation infrastructure only.

After Phase 5 (Weeks 12–13): Explainability & Adaptive Loop

- All Phase 4 files +
- configs/shap_config.yaml
- src/explainability/__init__.py, shap_analysis.py, adaptive_loop.py, visualize.py
- tests/test_explainability/test_shap.py, test_adaptive_loop.py
- notebooks/05_shap_explainability.ipynb

After Phase 6 (Weeks 14–16): Documentation & Finalization

- All Phase 5 files +
- configs/experiment_tracking.yaml
- src/benchmark/report.py
- .github/workflows/ci.yml, docs.yml
- .pre-commit-config.yaml
- docs/ (all files)
- notebooks/06_final_results.ipynb
- results/ (all populated subdirectories)
- scripts/download_models.sh

Section C — Chapter / Phase / Task / Sub-Task Breakdown

Chapter 1: Data Acquisition & Preprocessing Pipeline

Phase 1.1 — Raw Data Acquisition (Week 1)

Task 1.1.1 — Download 1000 Genomes Phase 3 Data

Sub-Task	Description	Status
1.1.1.1	Write `scripts/download_data.sh`	Not Started
1.1.1.2	Write `src/data/download.py`	Not Started
1.1.1.3	Simulate epistatic phenotype labels	Not Started

- ☐

  Sub-Task 1.1.1.1 — Write `scripts/download_data.sh`

  - **What to build:** Shell script `scripts/download_data.sh`
  - **Procedure:**
    1. Download pre-formatted PLINK2 files (`.pgen`, `.pvar`, `.psam`) from https://www.cog-genomics.org/plink/2.0/resources#1kg_phase3
    2. **Note:** The COG-Genomics resource provides a **single merged genome-wide file**, not per-chromosome files. Download the merged file directly.
    3. Use `wget` with `--continue` flag for resumability
    4. Compute and verify SHA256 checksums
    5. Store in `data/raw/1kg_phase3/`
  - **Input:** URLs (hardcoded in script)
  - **Output:** `data/raw/1kg_phase3/all_phase3.pgen`, `.pvar`, `.psam` files
  - **Acceptance criteria:** All files downloaded; checksums match; script is idempotent (re-running skips existing files)
  - **Assigned to: Mem1** (Genomics Lead)
  - **Depends on:** None

- ☐

  Sub-Task 1.1.1.2 — Write `src/data/download.py`

  - **What to build:** Python wrapper `download_1kg(output_dir)` in `src/data/download.py`
  - **Procedure:**
    1. Implement a Python function that calls `download_data.sh` via

`subprocess`

2. Add progress logging via `src/utils/logging.py`
3. Verify file existence and size after download
4. Return a dict mapping dataset → file path

- **Input:** `output_dir: str`
- **Output:** Dict `{"genome_wide": "data/raw/1kg_phase3/all_phase3.pgen", ...}`
- **Acceptance criteria:** Function completes without error; all files exist at returned paths; unit test passes with mocked subprocess
- **Assigned to: Mem1**
- **Depends on:** 1.1.1.1

- ☐

Sub-Task 1.1.1.3 — Write `src/data/simulate_phenotype.py` — Simulate Epistatic Phenotype

- **What to build:** Function `simulate_epistatic_phenotype(snp_matrix, interaction_pairs, betas, prevalence, seed)`
- **Procedure:**
  1. **CRITICAL DESIGN DECISION:** 1000 Genomes does NOT contain T2D phenotype labels. To avoid **circular validation** (Fault #1/14), phenotypes are generated from **injected epistatic interactions**, NOT from GWAS linear betas.
  2. Define a set of known/synthetic epistatic pair coefficients:

     ```
     liability = Σ(β_ij × SNP_i × SNP_j) + ε
     ```

     where `β_ij` are the interaction coefficients (NOT the same as PRS GWAS betas), and `ε ~ N(0, 1)`.
  3. **No linear PRS contribution** in the liability model — this ensures PRS is NOT trivially the best predictor, and the epistasis-detecting models have a genuine advantage.
  4. Cases = individuals with liability above a prevalence-based threshold (set to match ~10% T2D prevalence).
  5. Save phenotype file as `data/external/epistatic_phenotypes.csv` with columns: `sample_id`, `case_control`, `liability_score`
  6. Save the ground-truth interaction pairs + coefficients as `data/external/ground_truth_pairs.csv` — this enables

computing Epistasis Detection Rate (EDR).

- **Input:** `snp_matrix: np.ndarray (N, P)`, `interaction_pairs: List[(int, int)]`, `betas: np.ndarray`, `prevalence: float`, `seed: int`
- **Output:** `data/external/epistatic_phenotypes.csv`, `data/external/ground_truth_pairs.csv`
- **Acceptance criteria:** Phenotype file has one row per sample; case/control ratio ~10%; ground-truth pairs are documented; PRS computed from independent GWAS betas does NOT perfectly predict these labels (AUC < 0.7 for PRS alone)
- **Assigned to: Mem1**
- **Depends on:** 1.2.1.1 (needs SNP matrix first — but design can be done in Week 1)

Task 1.1.2 — Quality Control Pipeline

Sub-Task	Description	Status
1.1.2.1	Write `scripts/run_qc.sh`	Not Started
1.1.2.2	Write `src/data/qc.py`	Not Started

- ☐

Sub-Task 1.1.2.1 — Write `scripts/run_qc.sh`

- **What to build:** Shell script `scripts/run_qc.sh`
- **Procedure:**
  1. **Note:** Since 1000 Genomes Phase 3 from COG-Genomics is already a single merged genome-wide file, no `--pmerge-list` step is needed. Skip the merge step.
  2. Apply standard GWAS QC filters:
     - `--maf 0.01` (remove rare variants with MAF < 1%)
     - `--hwe 1e-6` (remove variants violating Hardy-Weinberg Equilibrium)
     - `--geno 0.05` (remove variants with >5% missing genotypes)
     - `--mind 0.1` (remove samples with >10% missing genotypes)
     - `--max-alleles 2` (keep only biallelic SNPs)
     - `--snps-only just-acgt` (exclude indels)
  3. Perform LD pruning: `--indep-pairwise 50 5 0.2`
  4. Extract pruned SNP set: `--extract prune.in`
  5. Export to PLINK2 binary format in `data/interim/`

6. Log variant counts before and after each step

- **Input:** `data/raw/1kg_phase3/` directory
- **Output:** `data/interim/qc_filtered.pgen`, `.pvar`, `.psam`; QC log file
- **Acceptance criteria:** Variant count reduced from ~80M to ~500K; all QC thresholds documented in log; script runs end-to-end in <30 min on HPC
- **Assigned to: Mem1**
- **Depends on:** 1.1.1.1

- ☐

  Sub-Task 1.1.2.2 — Write `src/data/qc.py`

  - **What to build:** Python wrapper `run_qc(input_dir, output_dir, config)` in `src/data/qc.py`
  - **Procedure:**
    1. Accept QC thresholds from YAML config (maf, hwe, geno, mind)
    2. Call `run_qc.sh` via subprocess with parameters
    3. Parse plink2 log output to extract variant/sample counts
    4. Return a `QCReport` dataclass with counts at each step
  - **Input:** `input_dir: str`, `output_dir: str`, `config: dict`
  - **Output:** `QCReport` dataclass; filtered PLINK files on disk
  - **Acceptance criteria:** QCReport has correct counts matching plink2 log; function is testable with mock data
  - **Assigned to: Mem1**
  - **Depends on:** 1.1.2.1

Phase 1.2 — SNP Matrix Construction (Week 2)

Task 1.2.1 — Preprocessing & Feature Engineering

Sub-Task	Description	Status
1.2.1.1	Write `src/data/preprocess.py`	Not Started
1.2.1.2	Write `src/data/epistasis_pairs.py`	Not Started
1.2.1.3	Write `src/data/dataset.py`	Not Started
1.2.1.4	Write EDA notebook	Not Started

- ☐

  Sub-Task 1.2.1.1 — Write `src/data/preprocess.py`

  - **What to build:** Functions `load_plink_to_matrix(plink_prefix)`, `impute_missing(matrix, strategy)`, `standardize(matrix)`, `select_top_snps(matrix, gwas_stats, n_snps)`
  - **Procedure:**
    1. `load_plink_to_matrix`: Use `plink2 --export A` (additive dosage) to convert binary PLINK to a `.raw` file, then read with `pd.read_csv("file.raw", sep="\t")`. Output shape: `(N_samples, N_variants)` with values in {0, 1, 2}. **No `pandas_plink` dependency needed.**
    2. `impute_missing`: Replace missing values (NaN / -9) with column median (most common for SNP data) or use `sklearn.impute.SimpleImputer`
    3. `standardize`: Apply `StandardScaler` — mean center and unit variance per SNP
    4. `select_top_snps`: Load GWAS summary stats, sort by p-value, select top `n_snps` (default 1000), return filtered matrix with SNP metadata
    5. Save processed matrix as `data/processed/snp_matrix.npy` and metadata as `data/processed/snp_metadata.csv`
  - **Input:** `plink_prefix: str`, `gwas_stats: str`, `n_snps: int`
  - **Output:** `np.ndarray` shape `(N, P)` dtype `float32`, metadata DataFrame
  - **Acceptance criteria:** No NaN values in output; shape matches (N_samples, n_snps); standardized columns have mean ≈ 0, std ≈ 1; unit test with synthetic 100×50 matrix passes
  - **Assigned to: Mem1**
  - **Depends on:** 1.1.2.2

- ☐

  Sub-Task 1.2.1.2 — Write `src/data/epistasis_pairs.py`

  - **What to build:** Function `select_epistasis_candidates(snp_matrix, gwas_stats, n_pairs, ld_threshold)`
  - **Procedure:**
    1. **Pair ranking method (Fault #17 fix):** Use **GWAS p-value product** as the ranking score: `score_pair(i, j) = -log10(p_i) × -log10(p_j)`. Pairs where both SNPs are individually significant are better epistasis candidates.
    2. Alternatively, use `plink2 --epistasis` (Fast-Epistasis) for regression-

based interaction testing if compute allows.

3. Filter out pairs where both SNPs are in high LD (`r² > ld_threshold`)
4. Sort by score descending, return top `n_pairs`
5. **Justification for n_pairs=500:** from 1000 SNPs there are ~499,500 possible pairs. We select top 500 (0.1%) as a tractable subset for quantum encoding. This number is chosen to balance (a) coverage of epistatic interactions and (b) VQC compute cost (125 circuits × 2504 samples = ~313K circuit evaluations per forward pass).
6. Return list of tuples `[(idx_a, idx_b, score), ...]` and save to `data/processed/epistasis_candidates.csv`

- **Input:** `snp_matrix: np.ndarray`, `gwas_stats: DataFrame`, `n_pairs: int` (default 500), `ld_threshold: float` (default 0.2)
- **Output:** `List[Tuple[int, int, float]]` — list of (snp_idx_a, snp_idx_b, interaction_score)
- **Acceptance criteria:** All pair indices within valid range; no self-pairs; no duplicate pairs; LD filter correctly removes correlated pairs; test with 10 synthetic SNPs yields expected ranking
- **Assigned to: Mem1**
- **Depends on:** 1.2.1.1

- ☐

  Sub-Task 1.2.1.3 — Write `src/data/dataset.py`

  - **What to build:** Class `SNPDataset(torch.utils.data.Dataset)` with mode support, and function `create_dataloaders(matrix, labels, batch_size, val_split, seed)`
  - **Procedure:**
    1. `SNPDataset.__init__`: accept `np.ndarray` matrix + labels; convert to `torch.Tensor`; accept `mode` parameter:
       - `mode='matrix'`: return full SNP feature row `(P,)` — for classical baselines
       - `mode='pairs'`: accept `pair_indices` list, return SNP pair values `(n_pairs, 2)` — for VQC
    2. `SNPDataset.__getitem__`: return `(features, label)` tuple based on mode
    3. `SNPDataset.__len__`: return number of samples
    4. `create_dataloaders`: split data into train/val/test (70/15/15) using `StratifiedShuffleSplit` (preserve class ratio); return three `DataLoader` objects

- **Input:** SNP matrix `(N, P)`, labels `(N,)`, batch_size, val_split, seed, mode, pair_indices
- **Output:** Three `DataLoader` objects (train, val, test)
- **Acceptance criteria:** Class ratios preserved across splits (within 2%); DataLoader iterates without error; batch shapes match expectations for both modes
- **Assigned to: Mem1**
- **Depends on:** 1.2.1.1

- ☐

  Sub-Task 1.2.1.4 — Write `notebooks/01_data_exploration.ipynb`

  - **What to build:** Jupyter notebook performing exploratory data analysis
  - **Procedure:**
    1. Load processed SNP matrix and phenotype labels
    2. Plot: sample size, variant count, class distribution (case vs control)
    3. Plot: MAF distribution histogram
    4. Plot: Missingness rate per SNP and per sample
    5. Plot: PCA of SNP matrix colored by population (1000 Genomes has AFR, AMR, EAS, EUR, SAS)
    6. Plot: Top-20 SNP pairs by interaction score
    7. Summary statistics table
  - **Input:** Processed data from `data/processed/`
  - **Output:** Completed notebook with inline plots
  - **Acceptance criteria:** All cells run without error; at least 6 publication-quality plots; narrative text explains each finding
  - **Assigned to: Mem1**
  - **Depends on:** 1.2.1.1, 1.2.1.2

Chapter 2: VQC Epistasis Encoder Design & Training

Phase 2.1 — Quantum Circuit Design (Week 3)

Week 3 sync checkpoint: Mem1 hands off `data/processed/snp_matrix.npy` and `epistasis_candidates.csv` to **Mem2**. Until this handoff, **Mem2** works with synthetic data.

Task 2.1.1 — SNP Pair Encoding Circuit

Sub-Task	Description	Status
2.1.1.1	Write `src/quantum/encoder.py`	Not Started
2.1.1.2	Write `src/quantum/vqc.py`	Not Started
2.1.1.3	Write unit tests for quantum circuits	Not Started

- ☐

  Sub-Task 2.1.1.1 — Write `src/quantum/encoder.py`

  - **What to build:** Function `encode_snp_pair(snp_a: float, snp_b: float, wires: List[int])`
  - **Procedure:**
    1. Map SNP dosage values {0, 1, 2} to angles: `angle = snp_value * π / 2` (maps $0 \to 0$, $1 \to \pi/2$, $2 \to \pi$)
    2. Apply `qml.RY(angle_a, wires=wires[0])` to encode first SNP
    3. Apply `qml.RY(angle_b, wires=wires[1])` to encode second SNP
    4. This creates a 2-qubit product state encoding the SNP pair
    5. Document the encoding scheme in docstring with mathematical notation
  - **Input:** `snp_a: float` (dosage 0/1/2), `snp_b: float` (dosage 0/1/2), `wires: List[int]`
  - **Output:** Quantum state on specified wires (no return value — modifies circuit)
  - **Acceptance criteria:** For input (0, 0), state is $|00\rangle$; for (2, 2), state is $|11\rangle$ (up to global phase); unit test verifies state vectors for all 9 dosage combinations
  - **Assigned to: Mem2** (Quantum Lead)
  - **Depends on:** None (uses processed data format spec from 1.2.1.1; tests with synthetic data first)

- ☐

  Sub-Task 2.1.1.2 — Write `src/quantum/vqc.py`

  - **What to build:** Class `VQCEpistasis(n_qubits: int = 8, n_layers: int = 6, n_pairs_per_circuit: int = 4)`
  - **Procedure:**
    1. **Architecture (Fault #2 fix):** Each 8-qubit circuit encodes **4 SNP pairs** (2 qubits per pair). For 500 total pairs, run **125 independent 8-qubit circuits** per sample. Concatenate all 125×8=1000 expectation values as the quantum

feature vector.

2. Initialize PennyLane device: `qml.device("default.qubit", wires=n_qubits)`
3. Define the ansatz as a QNode:

```
For each of the 4 SNP pairs in this circuit:
  a. Apply encode_snp_pair(snp_a, snp_b, wires=[2*i, 2*i+1])
Then for each layer l in range(n_layers):
  - Apply Ry(θ[l, q]) to each qubit q (trainable)
  - Apply CZ with LINEAR-LADDER connectivity:
    CZ(0,1), CZ(2,3), CZ(4,5), CZ(6,7)  [even layer]
    CZ(1,2), CZ(3,4), CZ(5,6)            [odd layer]
  - Apply Ry(φ[l, q]) to each qubit q (trainable)
Measure: qml.expval(qml.PauliZ(wire)) for each wire
```

4. **CZ connectivity (Fault #14 fix):** Linear-ladder is the default. The ladder alternates between even-pair and odd-pair CZ layers to ensure all adjacent qubits are entangled within 2 layers. A circular variant (adding CZ(7,0)) should also be implemented and benchmarked in the qubit-scaling experiment.
5. Total trainable parameters per circuit: `n_layers × n_qubits × 2` = 6 × 8 × 2 = 96
6. All 125 circuits **share the same weights** (weight-tied architecture) — this reduces total parameters to 96 and enables generalization across SNP pairs.
7. Wrap as a `torch.nn.Module` using `qml.qnn.TorchLayer`
8. Forward pass: accept SNP pair batch `(B, n_pairs_per_circuit, 2)`, return `(B, n_qubits)` for one circuit. Full forward: loop over 125 circuit calls, concatenate → `(B, 1000)`.

- **Input:** SNP pair batch `torch.Tensor` shape `(B, 4, 2)` per circuit call; full input `(B, 500, 2)` reshaped to 125 circuit calls
- **Output:** Full quantum feature vector `torch.Tensor` shape `(B, 1000)` dtype `float64`
- **Acceptance criteria:**
  1. Output shape for batch of 16, 500 pairs: `(16, 1000)`
  2. All output values in [-1, 1] (valid PauliZ expectations)
  3. Gradients flow: `loss.backward()` does not error
  4. Parameters count = 96 (shared across all 125 circuits)
- **Assigned to: Mem2**
- **Depends on:** 2.1.1.1

- ☐

Sub-Task 2.1.1.3 — Write unit tests for quantum circuits

- **What to build:** `tests/test_quantum/test_encoder.py` and `tests/test_quantum/test_vqc.py`
- **Procedure:**
  1. `test_encoder.py`:
     - Test all 9 dosage combinations for 2 SNPs → verify output state via `qml.state()`
     - Test angle mapping: dosage 0 → angle 0, dosage 2 → angle π
  2. `test_vqc.py`:
     - Test output shape for various batch sizes (single circuit: `(B, 8)`, full: `(B, 1000)`)
     - Test output range (all values in [-1, 1])
     - Test gradient computation: create loss, call backward, verify gradients are non-None
     - Test determinism: same input + same seed → same output
     - Test parameter count = 96
     - Test both linear-ladder and circular CZ connectivity
- **Input:** Synthetic SNP pair data
- **Output:** Passing pytest tests
- **Acceptance criteria:** All tests pass; coverage ≥ 90% for encoder.py and vqc.py
- **Assigned to: Mem2**
- **Depends on:** 2.1.1.1, 2.1.1.2

Phase 2.2 — Quantum Kernel Implementation (Week 4)

Task 2.2.1 — Quantum Kernel Matrix

Sub-Task	Description	Status
2.2.1.1	Write `src/quantum/kernel.py`	Not Started
2.2.1.2	Write `tests/test_quantum/test_kernel.py`	Not Started

- ☐

  Sub-Task 2.2.1.1 — Write `src/quantum/kernel.py`

  - **What to build:** Class `QuantumKernel(n_qubits: int, n_layers: int)` with methods `compute_kernel_entry(x1, x2)` and `compute_kernel_matrix(X)`

- **Procedure:**
    1. The kernel operates on a **reduced representation** of the quantum features. Use a PCA-reduced or single-circuit summary vector of shape `(N, 8)` as input (not the full 1000-dim vector, which would require an enormous circuit).
    2. Define the quantum feature map circuit using manual CZ + Ry gates (consistent with VQC ansatz):

```
@qml.qnode(dev, diff_method="parameter-shift")
def kernel_circuit(x1, x2, params):
    # Feature map for x1
    qml.AngleEmbedding(x1, wires=range(n_qubits))
    for l in range(n_layers):
        for q in range(n_qubits):
            qml.RY(params[l, q], wires=q)
        for q in range(0, n_qubits-1, 2):
            qml.CZ(wires=[q, q+1])
    # Adjoint of feature map for x2
    qml.adjoint(...)
    # Measure fidelity
    return qml.expval(qml.Projector([0]*n_qubits,
wires=range(n_qubits)))
```

    3. `compute_kernel_entry(x1, x2)`: return `|⟨φ(x2)|φ(x1)⟩|²`
    4. `compute_kernel_matrix(X)`: for dataset X with N samples, compute N×N kernel matrix. Exploit symmetry: only compute upper triangle + diagonal. Parallelize where possible.
    5. Add caching: store computed kernel matrix to disk with hash of inputs
    6. For N=2504, 8 qubits: this is ~3.1M circuit evaluations — expected runtime ~8.6 hours on `default.qubit`, ~3 hours on `lightning.qubit`
- **Input:** Feature vectors `X: np.ndarray` shape `(N, d)` where `d ≤ n_qubits`
- **Output:** Kernel matrix `K: np.ndarray` shape `(N, N)` dtype `float64`
- **Acceptance criteria:**
    1. K is symmetric: `K == K.T`
    2. K is positive semi-definite: all eigenvalues ≥ -1e-10
    3. Diagonal entries are 1.0 (or very close): `K[i,i] ≈ 1.0`
    4. K[i,j] ∈ [0, 1]
- **Assigned to: Mem2**
- **Depends on:** 2.1.1.2

- ☐

  Sub-Task 2.2.1.2 — Write kernel unit tests

    - **What to build:** `tests/test_quantum/test_kernel.py`
    - **Procedure:**

1. Test symmetry of computed kernel matrix
2. Test PSD property (eigenvalue check)
3. Test diagonal is ≈ 1.0
4. Test with identical inputs: K[i,j] = 1.0 when x_i == x_j
5. Test with small (10-sample) synthetic dataset for correctness

- **Input:** Synthetic feature vectors
- **Output:** Passing tests
- **Acceptance criteria:** All tests pass
- **Assigned to: Mem2**
- **Depends on:** 2.2.1.1

Phase 2.3 — VQC Training (Week 5)

Task 2.3.1 — Training Pipeline

Sub-Task	Description	Status
2.3.1.1	Write `src/quantum/train_vqc.py`	Not Started
2.3.1.2	Write VQC demo notebook	Not Started
2.3.1.3	Write quantum kernel demo notebook	Not Started

- ☐

  Sub-Task 2.3.1.1 — Write `src/quantum/train_vqc.py`

  - **What to build:** Function `train_vqc(model, train_loader, val_loader, config)` returning trained model + training log
  - **Procedure:**
    1. Initialize optimizer: `torch.optim.Adam(model.parameters(), lr=config["lr"])` (default lr=0.01)
    2. Loss function: `torch.nn.BCEWithLogitsLoss()` for binary classification
    3. Training loop:

```
for epoch in range(config["n_epochs"]):  # default 50
    for batch_x, batch_y in train_loader:
        optimizer.zero_grad()
        predictions = model(batch_x)  # VQC forward pass (125
circuits)
        loss = criterion(predictions, batch_y)
        loss.backward()  # parameter-shift rule via PennyLane
        optimizer.step()
```

```
    # Validation step
    val_loss, val_auc = evaluate(model, val_loader)
    log_metrics(epoch, train_loss, val_loss, val_auc)
    # Early stopping: if val_loss has not improved for 10
epochs, stop
```

4. Save best model weights to `results/models/vqc_best.pt`
5. Save training log to `results/logs/vqc_training.csv`
6. Log: epoch, train_loss, val_loss, val_auc, learning_rate, time_per_epoch

- **Input:** `VQCEpistasis` model, DataLoaders, config dict
- **Output:** Trained model, `TrainingLog` dataclass (losses, metrics per epoch)
- **Acceptance criteria:**
  1. Training loss decreases over epochs
  2. Val AUC > 0.5 (better than random on synthetic data)
  3. Model weights saved and loadable
  4. Training log has one row per epoch
  5. Early stopping triggers correctly
- **Assigned to: Mem2**
- **Depends on:** 2.1.1.2, 1.2.1.3

- ☐

  Sub-Task 2.3.1.2 — Write `notebooks/02_vqc_demo.ipynb`

  - **What to build:** Interactive notebook demonstrating VQC training
  - **Procedure:**
    1. Load a small subset of processed SNP data (100 samples, 10 SNP pairs)
    2. Instantiate VQC with 4 qubits, 3 layers (small for demo speed)
    3. Train for 20 epochs with visualization of loss curve
    4. Show circuit diagram using `qml.draw(circuit)`
    5. Visualize learned parameters as heatmap
    6. Predict on test set, show AUC-ROC curve
  - **Input:** Processed data subset
  - **Output:** Completed notebook
  - **Acceptance criteria:** Runs in < 5 minutes on CPU; all cells execute; circuit diagram renders
  - **Assigned to: Mem2**
  - **Depends on:** 2.3.1.1

- ☐

  Sub-Task 2.3.1.3 — Write `notebooks/03_quantum_kernel_demo.ipynb`

  - **What to build:** Interactive notebook demonstrating quantum kernel SVM
  - **Procedure:**
    1. Use small dataset (50 samples, 4 features)
    2. Compute quantum kernel matrix
    3. Visualize kernel matrix as heatmap
    4. Train SVM with `sklearn.svm.SVC(kernel='precomputed')`
    5. Compare to RBF kernel SVM
    6. Show decision boundaries (for 2D PCA projection)
  - **Input:** Processed data subset
  - **Output:** Completed notebook
  - **Acceptance criteria:** Runs in < 10 minutes; kernel matrix visualization correct; SVM trains successfully
  - **Assigned to: Mem2**
  - **Depends on:** 2.2.1.1

Chapter 3: Classical Baseline Models

Phase 3.1 — Baseline Implementation (Week 6)

Task 3.1.1 — PRS & Classical Models

Sub-Task	Description	Status
3.1.1.1	Write `src/classical/prs.py`	Not Started
3.1.1.2	Write `src/classical/baselines.py`	Not Started
3.1.1.3	Write `src/classical/metrics.py`	Not Started
3.1.1.4	Write classical baseline tests	Not Started

- ☐

  Sub-Task 3.1.1.1 — Write `src/classical/prs.py`

  - **What to build:** Function `compute_prs(snp_matrix, gwas_weights)` and

`evaluate_prs(prs_scores, labels)`

- **Procedure:**
  1. `compute_prs`:
     - Load GWAS effect sizes (beta coefficients) for each SNP — **these must be INDEPENDENT betas from a published GWAS (e.g., DIAGRAM), NOT the coefficients used in phenotype simulation**
     - PRS_i = Σ_j (β_j × dosage_ij) for each sample i
     - Handle missing GWAS weights: set to 0 and log a warning
     - Return PRS vector shape `(N,)`
  2. `evaluate_prs`:
     - Threshold PRS at various cutoffs to create binary predictions
     - Compute AUC-ROC of raw PRS scores vs labels
     - Return AUC, optimal threshold, classification report at optimal threshold
- **Input:** `snp_matrix: np.ndarray (N, P)`, `gwas_weights: np.ndarray (P,)`
- **Output:** `prs_scores: np.ndarray (N,)`, evaluation metrics dict
- **Acceptance criteria:** PRS computation is vectorized (no Python loops); AUC computation matches sklearn; test with known synthetic data where PRS should perfectly separate classes
- **Assigned to: Mem3** (ML/Analysis Lead)
- **Depends on:** 1.2.1.1

- ☐

Sub-Task 3.1.1.2 — Write `src/classical/baselines.py`

- **What to build:** Function `train_baselines(X_train, y_train, X_val, y_val, config)` returning dict of trained models
- **Procedure:**
  1. Train 4 classical models:
     - **Logistic Regression:** `sklearn.linear_model.LogisticRegression(C=1.0, max_iter=1000, solver='lbfgs')`
     - **Random Forest:** `sklearn.ensemble.RandomForestClassifier(n_estimators=500, max_depth=10, class_weight='balanced')`
     - **XGBoost:**

```
scale_pos_weight = (y_train == 0).sum() / (y_train ==
```

```
1).sum()
xgboost.XGBClassifier(n_estimators=500, max_depth=6,
learning_rate=0.01, scale_pos_weight=scale_pos_weight)
```

- **SVM (RBF):** `sklearn.svm.SVC(kernel='rbf', probability=True, class_weight='balanced')`

2. Hyperparameter tuning: use `sklearn.model_selection.GridSearchCV` with 3-fold inner CV
3. Evaluate each model on validation set (quick feedback — full CV deferred to Chapter 6)
4. Save each model with `joblib.dump` to `results/models/`
5. Return dict: `{"lr": model, "rf": model, "xgb": model, "svm": model}`

- **Input:** Train/val feature matrices + labels; config dict for hyperparameter grids
- **Output:** Dict of trained models; per-model metrics on validation set
- **Acceptance criteria:** All 4 models train without error; validation metrics logged; models are serializable
- **Assigned to: Mem3**
- **Depends on:** 1.2.1.1, 1.2.1.3

- ☐

Sub-Task 3.1.1.3 — Write `src/classical/metrics.py`

- **What to build:** Functions `compute_auc_roc(y_true, y_prob)`, `compute_f1(y_true, y_pred)`, `compute_epistasis_detection_rate(ranked_pairs, known_pairs, k)`, `compute_all_metrics(y_true, y_pred, y_prob, ranked_pairs, known_pairs)`
- **Procedure:**
  1. `compute_auc_roc`: wrapper around `sklearn.metrics.roc_auc_score` with error handling for single-class cases
  2. `compute_f1`: wrapper around `sklearn.metrics.f1_score` with `average='binary'`
  3. `compute_epistasis_detection_rate`:
     - **Novel metric definition:** EDR@K = |{known epistatic pairs in top-K ranked pairs}| / |{known epistatic pairs}|
     - Accept `ranked_pairs: List[(int,int)]` (model's ranking) and `known_pairs: List[(int,int)]` (ground truth from `data/external/ground_truth_pairs.csv`)

- Compute EDR at K=10, 20, 50, 100

4. `compute_all_metrics`: combine all above into a single dict

- **Input:** True labels, predicted labels/probabilities, ranked SNP pairs
- **Output:** Dict with keys: `auc_roc`, `f1`, `edr_10`, `edr_20`, `edr_50`, `edr_100`
- **Acceptance criteria:** AUC=1.0 for perfect predictions; F1=1.0 for perfect predictions; EDR=1.0 when top-K contains all known pairs; test with edge cases (all same class, all correct, all wrong)
- **Assigned to: Mem3**
- **Depends on:** None

- ☐

Sub-Task 3.1.1.4 — Write classical model tests

- **What to build:** `tests/test_classical/test_prs.py`, `test_baselines.py`, `test_metrics.py`
- **Procedure:**
  1. `test_prs.py`: test PRS = dot product for synthetic 5×3 matrix with known weights
  2. `test_baselines.py`: test all 4 models train on synthetic 100×20 data; test model saving/loading
  3. `test_metrics.py`: test all metrics with perfect, worst, and random predictions; test EDR edge cases
- **Input:** Synthetic data
- **Output:** Passing tests
- **Acceptance criteria:** All tests pass; coverage ≥ 85%
- **Assigned to: Mem3**
- **Depends on:** 3.1.1.1, 3.1.1.2, 3.1.1.3

Phase 3.2 — Baseline Validation (Week 7)

Task 3.2.1 — Validation Set Evaluation

Sub-Task	Description	Status
3.2.1.1	Evaluate all baselines on held-out validation set	Not Started
3.2.1.2	Generate baseline comparison table	Not Started

Note (Fault #10 fix): This phase runs validation-set evaluation only — NOT full 5-fold CV. Full CV is reserved for Chapter 6 to ensure all models are evaluated on identical

fold splits.

- ☐

  Sub-Task 3.2.1.1 — Run baseline validation evaluation

  - **What to build:** Script in `scripts/run_pipeline.py` (classical stage)
  - **Procedure:**
    1. Load processed SNP matrix and labels
    2. Use the fixed train/val/test split from `create_dataloaders` (70/15/15)
    3. Train all 4 baseline models + PRS on training set
    4. Evaluate on **validation set** (NOT test set — test set reserved for final evaluation)
    5. Collect AUC-ROC, F1, and training time
    6. Save results to `results/tables/baseline_val_results.csv`
  - **Input:** Processed data
  - **Output:** CSV with columns: model, auc_roc, f1, train_time_seconds
  - **Acceptance criteria:** 5 models evaluated; no NaN values; results give early quality feedback
  - **Assigned to: Mem3**
  - **Depends on:** 3.1.1.2

- ☐

  Sub-Task 3.2.1.2 — Generate baseline comparison table

  - **What to build:** Preliminary comparison table (full report in Chapter 6)
  - **Procedure:**
    1. Load validation results
    2. Generate bar chart comparing models
    3. Save plot to `results/figures/baseline_comparison_val.png`
  - **Input:** Validation results CSV
  - **Output:** PNG plot
  - **Acceptance criteria:** Plot has correct axis labels and legend
  - **Assigned to: Mem3**
  - **Depends on:** 3.2.1.1

Chapter 4: Quantum-Classical Hybrid Integration

Phase 4.1 — Feature Integration (Week 8)

Task 4.1.1 — Feature Concatenation & Hybrid Model

Sub-Task	Description	Status
4.1.1.1	Write `src/hybrid/feature_concat.py`	Not Started
4.1.1.2	Write `src/hybrid/model.py`	Not Started
4.1.1.3	Write `src/hybrid/train.py`	Not Started

- ☐

  Sub-Task 4.1.1.1 — Write `src/hybrid/feature_concat.py`

  - **What to build:** Function `concatenate_features(quantum_features, prs_scores, covariates=None)`
  - **Procedure:**
    1. Accept quantum features `(N, 1000)` from VQC output (125 circuits × 8 qubits)
    2. Accept PRS scores `(N, 1)` — from independent GWAS betas
    3. Optionally accept covariates `(N, k)` — population stratification PCs, etc.
    4. Concatenate along axis=1: result shape `(N, 1000 + 1 + k)`
    5. Optionally apply `StandardScaler` to quantum features to match classical feature scale
    6. Return combined feature matrix and feature name list
  - **Input:** Quantum `(N, 1000)`, PRS `(N, 1)`, covariates `(N, k)` or None
  - **Output:** Combined `np.ndarray` shape `(N, 1000+1+k)`, feature name list
  - **Acceptance criteria:** Output shape is sum of input feature dimensions; no NaN introduced; concatenation order is deterministic
  - **Assigned to: Mem2**
  - **Depends on:** 2.3.1.1, 3.1.1.1

- ☐

  Sub-Task 4.1.1.2 — Write `src/hybrid/model.py`

  - **What to build:** Class `HybridClassifier` with methods `fit(X, y, kernel_matrix)`, `predict(X, K_test)`, `predict_proba(X,`

`K_test)`

- **Procedure:**
    1. **Architecture: 2-level stacking ensemble**
        - Level 1 Model A: `sklearn.svm.SVC(kernel='precomputed')` using quantum kernel matrix
        - Level 1 Model B: `xgboost.XGBClassifier()` using concatenated features
        - Level 2 (Meta-learner): `sklearn.linear_model.LogisticRegression()` trained on Level 1 outputs
    2. `fit`:
        - Train SVC on quantum kernel matrix + labels
        - Train XGBoost on concatenated features + labels
        - Generate Level 1 predictions via cross-val to avoid information leakage
        - Stack Level 1 predictions as features for Level 2
        - Train LogisticRegression on stacked predictions
    3. `predict(X, K_test)`: run inputs through both Level 1 models, stack, predict with Level 2. `K_test` has shape `(N_test, N_train)` for the kernel SVC.
    4. `predict_proba(X, K_test)`: same as predict but return probabilities
    5. **Expose XGBoost sub-model** as `self.xgb_model` for SHAP analysis (Fault #6 fix)
    6. Save/load methods for model persistence
- **Input:** Feature matrix, labels, kernel matrix (and K_test for prediction)
- **Output:** Predictions `(N,)`, probabilities `(N, 2)`
- **Acceptance criteria:** Ensemble trains without error; predict_proba sums to 1.0; model is serializable with joblib; XGBoost sub-model accessible; test on synthetic data
- **Assigned to: Mem2** + **Mem3** (pair — **Mem2** owns quantum kernel integration, **Mem3** owns XGBoost + stacking. Coordinate via shared interface defined in Week 7.)
- **Depends on:** 2.2.1.1, 4.1.1.1

- ☐

Sub-Task 4.1.1.3 — Write `src/hybrid/train.py`

- **What to build:** Function `run_hybrid_pipeline(config)` — end-to-end hybrid training
- **Procedure:**
    1. Load processed data and epistasis pairs
    2. Generate VQC features for all SNP pairs (or load from cache)
    3. Compute quantum kernel matrix (or load from cache)
    4. Compute PRS scores (using independent GWAS betas)
    5. Concatenate features
    6. Train HybridClassifier
    7. Evaluate on test set
    8. Save all artifacts (model, predictions, metrics)
    9. Return metrics dict
- **Input:** Config dict (paths, hyperparameters)
- **Output:** Trained HybridClassifier, metrics dict, saved artifacts
- **Acceptance criteria:** Full pipeline runs end-to-end; all intermediate artifacts cached; metrics saved to `results/logs/`
- **Assigned to: Mem2**
- **Depends on:** 4.1.1.1, 4.1.1.2

Phase 4.2 — Integration Testing & Optimization (Weeks 9–10)

Task 4.2.1 — End-to-End Pipeline & Optimization

Sub-Task	Description	Status
4.2.1.1	Write `scripts/run_pipeline.py`	Not Started
4.2.1.2	Write integration test `tests/test_hybrid/test_model.py`	Not Started
4.2.1.3	Optimize quantum circuit for speed	Not Started
4.2.1.4	Write `notebooks/04_hybrid_pipeline.ipynb`	Not Started

- ☐

  Sub-Task 4.2.1.1 — Write `scripts/run_pipeline.py`

  - **What to build:** CLI entry point with stages: `data`, `vqc`, `classical`, `hybrid`,

`explain`, `benchmark`, `all`

- **Procedure:**
  1. Use `argparse` to accept:
     - `--stage`: which pipeline stage to run
     - `--config`: path to YAML config file
     - `--seed`: random seed
     - `--output-dir`: where to save results
  2. Dispatch to appropriate module based on stage
  3. Log start/end time, config used, and output paths
  4. Exit with code 0 on success, 1 on failure
- **Input:** CLI arguments
- **Output:** Pipeline outputs as specified by stage
- **Acceptance criteria:** `python scripts/run_pipeline.py --stage all --config configs/default.yaml` runs end-to-end; `--help` shows all options
- **Assigned to: Mem3**
- **Depends on:** 4.1.1.3

- ☐

  Sub-Task 4.2.1.2 — Write hybrid integration test

  - **What to build:** `tests/test_hybrid/test_model.py`
  - **Procedure:**
    1. Create synthetic dataset: 200 samples, 50 features, binary labels
    2. Create synthetic quantum features: 200×1000 random
    3. Create synthetic kernel matrix: 200×200 RBF kernel
    4. Train HybridClassifier on synthetic data
    5. Assert predictions have correct shape and valid values
    6. Assert probabilities sum to 1
    7. Assert AUC > 0.5 (better than random on non-trivial data)
    8. **Test kernel matrix subsetting:** verify that `K_test = K[np.ix_(test_idx, train_idx)]` has shape `(n_test, n_train)`
  - **Input:** Synthetic data
  - **Output:** Passing tests

- **Acceptance criteria:** All tests pass
- **Assigned to: Mem3**
- **Depends on:** 4.1.1.2

- ☐

  Sub-Task 4.2.1.3 — Optimize quantum circuit for speed

  - **What to build:** Performance optimizations in `src/quantum/` modules
  - **Procedure:**
    1. Profile VQC forward pass and kernel computation using `cProfile`
    2. Switch from `default.qubit` to `lightning.qubit` if available (recommended addition: faster C++ simulator)
    3. Batch quantum circuit evaluations where possible
    4. Cache kernel matrix entries that have been computed
    5. Implement parallel kernel computation using `multiprocessing` or `concurrent.futures`
    6. Document speedup achieved (target: 3-5x over naive implementation)
  - **Input:** Current quantum code
  - **Output:** Optimized quantum code; benchmark comparison (before/after)
  - **Acceptance criteria:** VQC forward pass for 100 samples completes in < 60 seconds on CPU; kernel matrix for 500 samples completes in < 30 minutes
  - **Assigned to: Mem2**
  - **Depends on:** 2.3.1.1, 2.2.1.1

  Recommended Addition: `pennylane-lightning` — PennyLane's C++-compiled high-performance simulator. Install via `pip install pennylane-lightning`. Provides 3-10x speedup over `default.qubit` for state-vector simulations.

- ☐

  Sub-Task 4.2.1.4 — Write hybrid pipeline demo notebook

  - **What to build:** `notebooks/04_hybrid_pipeline.ipynb`
  - **Procedure:**
    1. Load processed data (small subset)
    2. Show VQC feature extraction step by step
    3. Show quantum kernel computation with heatmap
    4. Show feature concatenation
    5. Train hybrid model, show predictions

6. Compare hybrid vs baseline metrics in a table
7. Plot ROC curves for all models on same axes

- **Input:** Processed data, trained models
- **Output:** Completed notebook
- **Acceptance criteria:** All cells run; ROC plot shows all models; narrative explains hybrid advantage/disadvantage
- **Assigned to: Mem3**
- **Depends on:** 4.1.1.3

Phase 4.3 — Benchmarking Infrastructure (Week 11)

Task 4.3.1 — Evaluation Framework

Sub-Task	Description	Status
4.3.1.1	Write `src/benchmark/evaluate.py`	Not Started
4.3.1.2	Write `src/benchmark/statistical_tests.py`	Not Started

- ☐

Sub-Task 4.3.1.1 — Write `src/benchmark/evaluate.py`

- **What to build:** Function `run_cross_validation(models_dict, X, y, kernel_matrix, n_folds, seed)`
- **Procedure:**
  1. Use `StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=seed)`
  2. For each fold:
     - Split X, y into train/test
     - **Kernel matrix subsetting (Fault #12 fix):**

```
K_train = kernel_matrix[np.ix_(train_idx, train_idx)]  # (n_train, n_train)
K_test = kernel_matrix[np.ix_(test_idx, train_idx)]    # (n_test, n_train)
```

     - Train each model in `models_dict` on training data (SVC uses K_train, others use X_train)
     - Evaluate on test data using `compute_all_metrics` (SVC uses K_test)
     - Store per-fold results

3. Aggregate results: compute mean, std, min, max across folds
4. Save per-fold results to CSV and aggregated results to separate CSV
5. Return `EvaluationReport` dataclass

- **Input:** Dict of models, features, labels, kernel matrix, n_folds
- **Output:** `EvaluationReport` with per-fold and aggregated metrics
- **Acceptance criteria:** All folds run; no data leakage between folds; kernel matrix correctly subsetted (verify shapes)
- **Assigned to: Mem3**
- **Depends on:** 3.1.1.3, 4.1.1.2

- ☐

  Sub-Task 4.3.1.2 — Write `src/benchmark/statistical_tests.py`

  - **What to build:** Functions `delong_test(y_true, proba_a, proba_b)`, `mcnemar_test(y_true, pred_a, pred_b)`, `bootstrap_ci(y_true, y_proba, n_boot, ci_level)`
  - **Procedure:**
    1. `delong_test`: Implement DeLong's test for comparing two AUC-ROC values. Use the `scipy.stats` framework. Return p-value and z-statistic.
    2. `mcnemar_test`: Compute McNemar's contingency table and chi-squared test. Use `statsmodels.stats.contingency_tables.mcnemar`. Return p-value.
    3. `bootstrap_ci`: Resample (with replacement) N times, compute AUC on each resample, return 95% CI from percentiles.
    4. All tests should handle edge cases: identical predictions, all correct, all wrong.
  - **Input:** True labels + predictions/probabilities from two models
  - **Output:** p-value, test statistic, confidence intervals
  - **Acceptance criteria:** Identical models → p-value ≈ 1.0; perfect vs random model → p-value ≈ 0.0; bootstrap CI contains true AUC with ~95% frequency
  - **Assigned to: Mem3**
  - **Depends on:** None (utility module)

Chapter 5: Explainability & Adaptive SNP Feedback Loop

Phase 5.1 — SHAP Analysis (Week 12)

Task 5.1.1 — SHAP Value Computation

Sub-Task	Description	Status
5.1.1.1	Write `src/explainability/shap_analysis.py`	Not Started
5.1.1.2	Write `src/explainability/visualize.py`	Not Started
5.1.1.3	Write SHAP unit tests	Not Started

- ☐

  Sub-Task 5.1.1.1 — Write `src/explainability/shap_analysis.py`

  - **What to build:** Functions `compute_shap_values(xgb_model, X)` and `rank_snp_pairs(shap_values, pair_indices)`
  - **Procedure:**
    1. `compute_shap_values`:
       - **SHAP is computed ONLY on the XGBoost sub-model** within the hybrid ensemble (Fault #6 fix). This avoids the $O(N^2)$ kernel computation per SHAP sample that would make KernelExplainer intractable.
       - Use `shap.TreeExplainer(hybrid_model.xgb_model)` — fast, exact, O(TLD)
       - **Limitation (document explicitly):** SHAP values rank features for the classical XGBoost component, not the quantum kernel component. The adaptive loop uses this to select which SNP pairs are worth re-encoding in the VQC.
       - Return SHAP values array `(N, P)` and base value
    2. `rank_snp_pairs`:
       - Map individual SNP SHAP values to pair-level importance
       - For each pair (i, j): `pair_importance = mean(|SHAP_i|) + mean(|SHAP_j|)`
       - If SHAP interaction values available (from TreeExplainer): add `mean(|SHAP_interaction_ij|)` — but note this is $O(N \times P^2)$, use only for top-50 pairs
       - Sort pairs by pair_importance descending

        - Return ranked list
    3. Save SHAP values to `results/shap_values.npy`
    4. Save ranking to `results/tables/snp_pair_ranking.csv`
- **Input:** Trained XGBoost sub-model, feature matrix, pair index list
- **Output:** SHAP values `(N, P)`, ranked pair list
- **Acceptance criteria:** SHAP values have same shape as input features; ranking is deterministic for same input; top-ranked pairs have highest importance; computation completes in < 10 minutes for 1000 samples
- **Assigned to: Mem3**
- **Depends on:** 4.1.1.2

- ☐

  Sub-Task 5.1.1.2 — Write `src/explainability/visualize.py`

  - **What to build:** Functions `plot_shap_summary(shap_values, feature_names, save_path)`, `plot_shap_waterfall(shap_explanation, sample_idx, save_path)`, `plot_snp_pair_importance(ranking, top_k, save_path)`
  - **Procedure:**
    1. `plot_shap_summary`: use `shap.summary_plot` — show top 20 features, beeswarm plot
    2. `plot_shap_waterfall` **(Fault #15 fix):** use the current SHAP API:

    ```
    shap.plots.waterfall(shap.Explanation(
        values=shap_values[sample_idx],
        base_values=explainer.expected_value,
        data=X[sample_idx],
        feature_names=feature_names
    ))
    ```

    3. `plot_snp_pair_importance`: horizontal bar chart of top-K SNP pairs by importance
    4. All plots: use publication-quality settings (DPI=300, readable font sizes, proper labels)
    5. Save all plots to `results/figures/`
  - **Input:** SHAP values, feature names, save paths
  - **Output:** PNG/SVG plot files
  - **Acceptance criteria:** All plots render without error; plots are publication quality; file sizes reasonable (< 5MB each)
  - **Assigned to: Mem3**

- **Depends on:** 5.1.1.1

- ☐

  Sub-Task 5.1.1.3 — Write SHAP unit tests

  - **What to build:** `tests/test_explainability/test_shap.py`
  - **Procedure:**
    1. Train a simple XGBoost on synthetic data
    2. Compute SHAP values → verify shape matches features
    3. Test ranking function → verify descending order
    4. Test with 2 features where one is clearly more important → verify it ranks first
  - **Input:** Synthetic data
  - **Output:** Passing tests
  - **Acceptance criteria:** All tests pass
  - **Assigned to: Mem3**
  - **Depends on:** 5.1.1.1

Phase 5.2 — Adaptive Feedback Loop (Week 13)

Task 5.2.1 — Iterative Re-Ranking & Retraining

Sub-Task	Description	Status
5.2.1.1	Write `src/explainability/adaptive_loop.py`	Not Started
5.2.1.2	Write adaptive loop tests	Not Started
5.2.1.3	Write explainability demo notebook	Not Started

- ☐

  Sub-Task 5.2.1.1 — Write `src/explainability/adaptive_loop.py`

  - **What to build:** Function `run_adaptive_loop(vqc, xgb_model, data, initial_pairs, config)` — the core innovation
  - **Procedure:**
    1. **Iteration 0:** Use initial SNP pairs from GWAS-based selection (from `epistasis_pairs.py`)
    2. **For each iteration i in range(config["max_iterations"]):** (max_iterations ≤ 3) a. **Warm-start VQC** from previous iteration's weights (Fault #28 fix) — do NOT reset weights b. Train VQC on current SNP pairs → get quantum

features `(N, 1000)` c. Concatenate with PRS + covariates → train XGBoost on combined features d. **No kernel recomputation during adaptive loop** (Fault #13 fix) — the adaptive loop uses VQC features + XGBoost only, dropping the kernel SVC for this phase. Kernel SVC is only used in the final non-adaptive hybrid model. e. Compute SHAP values on XGBoost (TreeExplainer — fast) f. Re-rank SNP pairs using SHAP values g. Replace bottom 20% of current pairs (100 pairs) with top 20% from newly-ranked pairs not currently in the set h. Log: iteration, pairs changed, AUC before/after, top-10 pairs i. **Convergence check:** if AUC improvement < 0.005, stop early

3. Return final VQC, final XGBoost model, iteration log, final SNP pair set
4. After adaptive loop converges: retrain the full hybrid model (including kernel SVC) ONE TIME on the final pair set for the definitive model
5. Save iteration log to `results/logs/adaptive_loop_log.csv`

- **Input:** VQC model, XGBoost model, data dict, initial pairs, config
- **Output:** Final VQC (retrained), final models, `AdaptiveLoopLog` with per-iteration metrics
- **Acceptance criteria:**
  1. Loop runs for ≤ 3 iterations or converges early
  2. Each iteration's SNP pair set differs from previous (unless converged)
  3. AUC is logged at each iteration
  4. Final model is at least as good as iteration 0 model (or within 0.01)
  5. All intermediate models and logs saved
  6. Warm-starting verified: iteration 1 starts from iteration 0 weights
- **Assigned to: Mem3** + **Mem2** (pair — **Mem2** handles VQC retraining, **Mem3** handles SHAP ranking)
- **Depends on:** 5.1.1.1, 2.3.1.1, 4.1.1.3

- ☐

  Sub-Task 5.2.1.2 — Write adaptive loop tests

  - **What to build:** `tests/test_explainability/test_adaptive_loop.py`
  - **Procedure:**
    1. Use tiny synthetic data (50 samples, 10 features, 5 pairs)
    2. Use small VQC (4 qubits, 1 layer, 2 pairs per circuit) for speed
    3. Run adaptive loop for 2 iterations
    4. Verify: iteration log has 2 entries; pair sets change; AUC is logged

5. Verify convergence: if injected early convergence, loop stops
6. Verify warm-starting: VQC weights at iteration 1 start != random initialization

- **Input:** Synthetic data, small models
- **Output:** Passing tests
- **Acceptance criteria:** Test runs in < 2 minutes; all assertions pass
- **Assigned to: Mem3**
- **Depends on:** 5.2.1.1

- ☐

  Sub-Task 5.2.1.3 — Write `notebooks/05_shap_explainability.ipynb`

  - **What to build:** Demo notebook for SHAP + adaptive loop
  - **Procedure:**
    1. Load pre-trained hybrid model
    2. Compute and visualize SHAP values (on XGBoost sub-model)
    3. Show SNP pair ranking
    4. Run 1 iteration of adaptive loop (live demo)
    5. Compare before/after rankings
    6. Discuss biological interpretation of top SNP pairs
    7. **Explicitly note limitation:** SHAP is on XGBoost only, not the full quantum hybrid
  - **Input:** Pre-trained models, processed data
  - **Output:** Completed notebook
  - **Acceptance criteria:** All cells run; clear visualizations; narrative explains the adaptive mechanism and its limitations
  - **Assigned to: Mem3**
  - **Depends on:** 5.2.1.1

Chapter 6: Benchmarking, Evaluation & Statistical Analysis

Phase 6.1 — Comprehensive Evaluation (Weeks 14–15, concurrent with docs)

Task 6.1.1 — Full Benchmark Suite

Sub-Task	Description	Status
6.1.1.1	Run full 5-fold CV for all models	Not Started
6.1.1.2	Run statistical significance tests	Not Started
6.1.1.3	Write `src/benchmark/report.py`	Not Started
6.1.1.4	Generate all result artifacts	Not Started
6.1.1.5	Run qubit-scaling experiment	Not Started

- ☐

  Sub-Task 6.1.1.1 — Run full 5-fold cross-validation

  - **What to build:** Execution of `run_cross_validation` with all models — **this is the ONLY full CV in the project** (Fault #10 fix)
  - **Procedure:**
    1. Models to evaluate (8 models, not 9):
       - PRS only (baseline)
       - Logistic Regression
       - Random Forest
       - XGBoost
       - SVM (RBF kernel)
       - VQC only (quantum features → logistic regression)
       - Quantum Kernel SVC
       - Hybrid (no adaptive loop)
    2. **Hybrid + Adaptive Loop:** Evaluated on a **fixed held-out test split** (Fault #20 fix), NOT inside CV. Running the adaptive loop inside each CV fold would require 5 × 3 = 15 full adaptive iterations and is computationally prohibitive. This limitation must be disclosed in the paper.
    3. Run 5-fold stratified CV for non-adaptive models with `random_state=42`
    4. Record: AUC-ROC, F1, EDR@K for K=10,20,50,100
    5. Record per-fold results and aggregates

6. Save to `results/tables/full_cv_results.csv`
7. **Note on kernel SVC in CV (Fault #18 fix):** each fold requires kernel matrix subsetting (~30 minutes per fold on `lightning.qubit`). Budget ~2.5 hours total for 5 folds.

- **Input:** Processed data, all model configs
- **Output:** Comprehensive results CSV
- **Acceptance criteria:** 8 models × 5 folds = 40 result rows for CV; 1 additional row for Hybrid+Adaptive on fixed test split; no missing values; results reproducible
- **Assigned to: Mem3**
- **Depends on:** 5.2.1.1, 3.1.1.2

- ☐

  Sub-Task 6.1.1.2 — Run statistical significance tests

  - **What to build:** Pairwise statistical comparisons
  - **Procedure:**
    1. For the primary comparison (Hybrid vs best classical baseline):
       - DeLong test on AUC-ROC
       - McNemar test on predictions
       - Bootstrap 95% CI for AUC difference
    2. Report p-values corrected for multiple testing (Bonferroni)
    3. For all model pairs: create a comparison matrix
    4. Save to `results/tables/statistical_tests.csv`
  - **Input:** Cross-validation predictions from all models
  - **Output:** Statistical test results CSV
  - **Acceptance criteria:** p-values in [0, 1]; Bonferroni correction applied; results table has all pairwise comparisons
  - **Assigned to: Mem3**
  - **Depends on:** 6.1.1.1, 4.3.1.2

- ☐

  Sub-Task 6.1.1.3 — Write `src/benchmark/report.py`

  - **What to build:** Function `generate_report(cv_results, stat_tests, output_dir)`
  - **Procedure:**
    1. Generate LaTeX table: models × metrics with bold for best values
    2. Generate ROC curve plot: all models on same axes, with AUC in legend

3. Generate bar chart: grouped bars for AUC and F1 per model
4. Generate adaptive loop convergence plot: AUC vs iteration
5. Generate SHAP summary for best model
6. Generate EDR@K curve: EDR vs K for top models
7. All plots: publication quality (DPI=300, sans-serif font, proper legend)
8. Save all to `results/figures/` and `results/tables/`

- **Input:** CV results DataFrame, statistical test results
- **Output:** LaTeX tables, PNG plots
- **Acceptance criteria:** All figures render; LaTeX compiles; plots have proper formatting
- **Assigned to: Mem3**
- **Depends on:** 6.1.1.1, 6.1.1.2

- ☐

  Sub-Task 6.1.1.4 — Generate all result artifacts

  - **What to build:** Run report generation and verify all outputs
  - **Procedure:**
    1. Execute `generate_report` with all results
    2. Verify all expected files exist in `results/`
    3. Create `results/README.md` documenting every file
    4. Verify LaTeX tables compile
    5. Create final results summary notebook `notebooks/06_final_results.ipynb`
  - **Input:** All model results
  - **Output:** Complete results directory
  - **Acceptance criteria:** `results/figures/` contains ≥ 6 plots; `results/tables/` contains ≥ 3 tables; final notebook runs end-to-end
  - **Assigned to: Mem3**
  - **Depends on:** 6.1.1.3

- ☐

  Sub-Task 6.1.1.5 — Run qubit-scaling experiment (Fault #18 fix)

  - **What to build:** Experiment using `configs/vqc_scaling.yaml` — train VQC at 4, 6, 8, 10 qubits
  - **Procedure:**

1. For each qubit count in {4, 6, 8, 10}:
    - Configure VQC with corresponding n_qubits and n_pairs_per_circuit
    - Train on the same data split
    - Record: AUC, training time, memory usage, gradient norm
2. Plot: AUC vs n_qubits, runtime vs n_qubits
3. This addresses the removed `vqc_10qubit.yaml` orphan config and provides a strong paper figure

- **Input:** Processed data, scaling config
- **Output:** Scaling results CSV + plots
- **Acceptance criteria:** 4 data points (one per qubit count); clear trend in runtime; plot is publication quality
- **Assigned to: Mem2**
- **Depends on:** 2.3.1.1

Chapter 7: Documentation, Paper, and Repository Finalization

Phase 7.1 — Documentation (Weeks 14–15, parallel with benchmarking)

Task 7.1.1 — Technical Documentation

Sub-Task	Description	Status
7.1.1.1	Write MkDocs configuration + index	Not Started
7.1.1.2	Write setup guide	Not Started
7.1.1.3	Write architecture documentation	Not Started
7.1.1.4	Write VQC design document	Not Started
7.1.1.5	Write API reference (auto-generated)	Not Started

- ☐

  Sub-Task 7.1.1.1 — Write `docs/mkdocs.yml` and `docs/index.md`

  - **Assigned to: Mem1**
  - **Depends on:** All prior chapters

- ☐

  Sub-Task 7.1.1.2 — Write `docs/setup_guide.md`

  - Prerequisites: Python 3.10+, plink2, Git

- **Assigned to: Mem1**
- **Depends on:** All code complete

- ☐

  Sub-Task 7.1.1.3 — Write `docs/architecture.md`

  - **Assigned to: Mem1**
  - **Depends on:** All code complete

- ☐

  Sub-Task 7.1.1.4 — Write `docs/vqc_design.md`

  - Must document: CZ linear-ladder connectivity choice and comparison with circular variant; the 4-pairs-per-8-qubit-circuit design; weight-tying rationale
  - **Assigned to: Mem2**
  - **Depends on:** VQC code complete

- ☐

  Sub-Task 7.1.1.5 — Write API reference (auto-generated)

  - **Assigned to: Mem1**
  - **Depends on:** All code complete with docstrings

Phase 7.2 — Paper & Finalization (Week 16)

Task 7.2.1 — Academic Paper & Repository Cleanup

Sub-Task	Description	Status
7.2.1.1	Write paper draft	Not Started
7.2.1.2	Write README.md	Not Started
7.2.1.3	Set up CI/CD	Not Started
7.2.1.4	Final repository review	Not Started

- ☐

  Sub-Task 7.2.1.1 — Write paper draft

  - **Target venue (Fault #19 fix):** IEEE BIBM (Bioinformatics and Biomedicine) conference. Use the IEEE conference LaTeX template from day one. Alternatively, publish as an **arXiv preprint** first with a future journal submission.
  - **Assigned to: Mem1** (lead) + all team
  - **Depends on:** 6.1.1.4

- ☐

  Sub-Task 7.2.1.2 — Write comprehensive `README.md`

- **Assigned to: Mem3** (Fault #11 fix — redistributed from Mem1)
- **Depends on:** All code + results complete

- ☐

  Sub-Task 7.2.1.3 — Set up CI/CD

  - Python version in CI: `3.10`
  - **Assigned to: Mem2** (Fault #11 fix — redistributed from Mem1)
  - **Depends on:** All code complete

- ☐

  Sub-Task 7.2.1.4 — Final repository review checklist

  - **Assigned to:** All team members (joint review)
  - **Depends on:** All prior sub-tasks

Section D — 3-Person Team Assignment

D.1 — Role Definitions

Person 1: Mem1 — Genomics & Data Engineering Lead

Role Definition: Mem1 owns the entire data pipeline from raw genomic data to processed feature matrices. They are the domain expert in bioinformatics and responsible for ensuring the genomic data is correctly acquired, quality-controlled, and prepared for downstream quantum and classical models. They also lead documentation and the academic paper.

Primary Responsibilities:

- Data acquisition (1000 Genomes download, epistatic phenotype simulation)
- GWAS QC pipeline (plink2 scripting)
- SNP matrix construction and preprocessing
- Epistasis candidate pair selection
- Documentation (MkDocs, setup guides, API reference)
- Academic paper (lead author on Methods/Data sections)

Skill Requirements:

- Bioinformatics: VCF format, PLINK2, GWAS concepts, LD, MAF, HWE
- Python data engineering: Pandas, NumPy, data pipeline design
- Technical writing: LaTeX, MkDocs

Handoff Points: | Handoff | Format | Recipient | |---|---|---| | Processed SNP matrix → VQC | `data/processed/snp_matrix.npy` shape `(N, P)` float32 | **Mem2** | | Epistasis pairs →

VQC | `data/processed/epistasis_candidates.csv` | **Mem2** | | Processed SNP matrix → Baselines | Same as above | **Mem3** | | Phenotype labels → All | `data/external/epistatic_phenotypes.csv` | **Mem2**, **Mem3** |

Person 2: Mem2 — Quantum Systems Engineer

Role Definition: Mem2 owns all quantum computing components — from circuit design to training. They are responsible for the VQC Epistasis Encoder, quantum kernel implementation, and quantum-classical integration layer.

Primary Responsibilities:

- VQC architecture design and implementation (PennyLane)
- SNP pair encoding circuit
- Quantum kernel matrix computation
- VQC training pipeline (parameter-shift rule + PyTorch)
- Quantum circuit optimization (speed, depth)
- Hybrid model architecture (feature concatenation + stacking)
- VQC design documentation
- CI/CD setup (redistributed from Mem1 for workload balance)

Skill Requirements:

- Quantum computing: qubits, gates (Ry, CZ), measurement, entanglement
- PennyLane: QNode, TorchLayer, device management, diff_method
- PyTorch: nn.Module, autograd, DataLoader, training loops
- Linear algebra: state vectors, unitary matrices, kernel theory

Handoff Points: | Handoff | Format | Recipient | |---|---|---| | VQC features → Hybrid model | `np.ndarray` shape `(N, 1000)` float64 | **Mem3** | | Kernel matrix → Hybrid model | `np.ndarray` shape `(N, N)` float64 | **Mem3** | | Trained VQC weights → Adaptive loop | `results/models/vqc_best.pt` | **Mem3** | | VQC design doc → Paper | `docs/vqc_design.md` | **Mem1** |

Person 3: Mem3 — ML/Analysis & Explainability Lead

Role Definition: Mem3 owns the classical ML pipeline, explainability (SHAP), the adaptive feedback loop, and all benchmarking/evaluation. They ensure that the quantum model's advantage (if any) is rigorously quantified.

Primary Responsibilities:

- Classical baseline models (LR, RF, XGBoost, SVM)

- PRS computation and evaluation
- Evaluation metrics (AUC-ROC, F1, EDR)
- SHAP explainability analysis (on XGBoost sub-model)
- Adaptive SNP feedback loop (with Mem2 for VQC retraining)
- Benchmarking framework (CV, statistical tests)
- Results generation (plots, tables, LaTeX)
- CLI pipeline script
- README writing (redistributed from Mem1 for workload balance)

Skill Requirements:

- Classical ML: sklearn, XGBoost, model selection, cross-validation
- SHAP: TreeExplainer, interpretation
- Statistics: DeLong test, McNemar test, bootstrap, multiple testing correction
- Visualization: Matplotlib, Seaborn, publication-quality plots

Handoff Points: | Handoff | Format | Recipient | |---|---|---| | Baseline metrics → Benchmark | `results/tables/baseline_val_results.csv` | **Mem1** (paper) | | SHAP rankings → Adaptive loop → VQC | `List[(int, int)]` top-K pairs | **Mem2** (VQC retrain) | | All result artifacts → Paper | `results/figures/`, `results/tables/` | **Mem1** (paper) |

D.2 — Gantt-Style Work Assignment (16 Weeks)

Week	Mem1 (Genomics/Docs)	Mem2 (Quantum)	Mem3 (ML/Analysis)
1	Download 1KG data (1.1.1.1, 1.1.1.2)	Study PennyLane tutorials, design circuit architecture on paper	Write `metrics.py` (3.1.1.3), set up utils
2	QC pipeline (1.1.2), start preprocess	Continue PennyLane study, prototype encoder with synthetic data	Write `prs.py` (3.1.1.1), `dataset.py` (1.2.1.3)
3	Preprocess (1.2.1.1), Epistasis pairs (1.2.1.2), Phenotype sim (1.1.1.3), **HANDOFF to Mem2**	Implement encoder (2.1.1.1) + VQC (2.1.1.2) with real data from Mem1	Write `baselines.py` (3.1.1.2)
4	EDA notebook (1.2.1.4), data documentation	Quantum kernel (2.2.1.1, 2.2.1.2), encoder tests (2.1.1.3)	Classical model tests (3.1.1.4)
5	Buffer / assist quantum team with data issues	VQC training (2.3.1.1), VQC demo notebook	Baseline validation eval (3.2.1.1),

Week	Mem1 (Genomics/Docs)	Mem2 (Quantum)	Mem3 (ML/Analysis)
		(2.3.1.2)	comparison (3.2.1.2)
6	Buffer / start docs framework (7.1.1.1)	Kernel demo notebook (2.3.1.3)	Baseline evaluation complete
7	Start setup guide (7.1.1.2)	Feature concatenation (4.1.1.1), coordinate hybrid interface with Mem3	Baseline comparison chart
8	Docs: architecture (7.1.1.3)	Hybrid model (4.1.1.2), Hybrid train (4.1.1.3)	Integration test (4.2.1.2), CLI script (4.2.1.1)
9	Continue docs	Optimize quantum circuits (4.2.1.3)	Hybrid pipeline notebook (4.2.1.4)
10	Continue docs	Optimization benchmarks	CV evaluate (4.3.1.1), stat tests (4.3.1.2)
11	API reference (7.1.1.5)	Help with benchmarking, SLURM scripts	SHAP analysis (5.1.1.1), visualize (5.1.1.2)
12	Continue API docs	Support VQC retraining in adaptive loop	SHAP tests (5.1.1.3), adaptive loop (5.2.1.1)
13	VQC design doc (7.1.1.4) — with Mem2	Adaptive loop VQC retraining + qubit-scaling experiment (6.1.1.5)	Adaptive loop tests (5.2.1.2), explainability notebook (5.2.1.3)
14	Paper draft — intro, methods, data (7.2.1.1)	CI/CD setup (7.2.1.3), paper VQC methods section	Full CV benchmark (6.1.1.1), stat tests (6.1.1.2)
15	Paper draft — results, discussion	Paper review + edits	Report generation (6.1.1.3), result artifacts (6.1.1.4)
16	Paper finalization, final review	Final review, circuit diagrams for paper	README (7.2.1.2), final results notebook, final review

Week 15 — Team-wide paper review session: all three members review the paper draft together.

Parallel Work Opportunities

Weeks	Parallel Streams
1–2	**Mem1** (data pipeline) ‖ **Mem2** (PennyLane study + design) ‖ **Mem3** (metrics + PRS)

Weeks	Parallel Streams
3–5	**Mem1** (preprocess + phenotype) ‖ **Mem2** (VQC + kernel) ‖ **Mem3** (baselines + validation)
6–7	**Mem1** (docs framework) ‖ **Mem2** (kernel demo + hybrid interface) ‖ **Mem3** (baseline report)
8–10	**Mem1** (docs) ‖ **Mem2** (hybrid model + optimization) ‖ **Mem3** (integration + benchmarking)
11–13	**Mem1** (API docs) ‖ **Mem2** (adaptive loop VQC + scaling) ‖ **Mem3** (SHAP + adaptive loop)
14–16	**Mem1** (paper) ‖ **Mem2** (CI/CD + paper section) ‖ **Mem3** (final results + README)

Section E — Tools Reference Guide

Tool: PennyLane

Version to use: `0.40.0` (or latest stable) **Install command:** `pip install pennylane==0.40.0` **Used in:** Phase 2 (all tasks), Phase 4 (hybrid integration), Phase 5 (adaptive loop VQC retraining) **Purpose in QGenomix:** Design, simulate, and train the VQC Epistasis Encoder; compute quantum kernel matrices; differentiate quantum circuits via the parameter-shift rule.

How to Use It

1. Create a device (simulator):

```
import pennylane as qml
dev = qml.device("default.qubit", wires=8)
```

1. Define a QNode for the VQC (consistent with CZ linear-ladder design):

```
@qml.qnode(dev, diff_method="parameter-shift", interface="torch")
def vqc_circuit(inputs, weights):
    # Angle encoding of 4 SNP pairs (8 values → 8 qubits)
    for i in range(4):
        qml.RY(inputs[2*i] * np.pi / 2, wires=2*i)
        qml.RY(inputs[2*i+1] * np.pi / 2, wires=2*i+1)
    # Trainable ansatz layers with CZ linear-ladder connectivity
    for l, layer_weights in enumerate(weights):
        for q in range(8):
            qml.RY(layer_weights[q], wires=q)
        if l % 2 == 0:  # even layer: CZ on (0,1),(2,3),(4,5),(6,7)
            for q in range(0, 8, 2):
                qml.CZ(wires=[q, q+1])
        else:  # odd layer: CZ on (1,2),(3,4),(5,6)
            for q in range(1, 7, 2):
                qml.CZ(wires=[q, q+1])
```

```
    return [qml.expval(qml.PauliZ(w)) for w in range(8)]
```

1. Wrap as PyTorch layer:

```
weight_shapes = {"weights": (n_layers, n_qubits)}
qlayer = qml.qnn.TorchLayer(vqc_circuit, weight_shapes)
```

Gotchas & Constraints

- `default.qubit` simulates full state vector: memory = O(2^n_qubits). For 12 qubits = 4096 complex128 entries = ~65 KB per state vector. During batched execution with N=2504 samples, total memory is 2504 × 65 KB ≈ **163 MB** — manageable. For 20+ qubits, memory explodes.
- Parameter-shift rule requires 2 circuit evaluations per parameter per gradient step. For 96 parameters: 192 circuit evaluations per backward pass.
- `interface="torch"` is required for PyTorch autograd integration.
- Batch execution: PennyLane does NOT natively batch over circuits. Use a for-loop or `qml.batch_input` decorator.
- **Python version:** PennyLane 0.40 requires Python ≥ 3.10 (recommended for performance). Python 3.9 is technically supported by PennyLane ≥0.38 but 3.10+ is recommended for this project. Check `qml.about()` to verify.

Tool: PyTorch

Version to use: `2.5.1` **Install command:** `pip install torch==2.5.1` **Used in:** Phase 2 (VQC training), Phase 4 (hybrid model training) **Purpose in QGenomix:** Provides autograd backend for VQC training via PennyLane's TorchLayer; serves as the optimizer and DataLoader framework.

How to Use It

1. Create DataLoader:

```
from torch.utils.data import DataLoader, TensorDataset

dataset = TensorDataset(torch.tensor(X, dtype=torch.float32),
                        torch.tensor(y, dtype=torch.float32))
loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

1. Training loop with PennyLane QNode:

```
optimizer = torch.optim.Adam(qlayer.parameters(), lr=0.01)
criterion = torch.nn.BCEWithLogitsLoss()

for epoch in range(50):
    for batch_x, batch_y in loader:
        optimizer.zero_grad()
        pred = qlayer(batch_x)
        loss = criterion(pred.squeeze(), batch_y)
        loss.backward()  # triggers parameter-shift via PennyLane
```

```
        optimizer.step()
```

Example Code Snippet

```
import torch
import torch.nn as nn

class HybridNet(nn.Module):
    def __init__(self, qlayer, classical_dim, n_quantum_outputs):
        super().__init__()
        self.qlayer = qlayer  # PennyLane TorchLayer
        self.classical = nn.Linear(classical_dim, 16)
        self.head = nn.Linear(16 + n_quantum_outputs, 1)  # n_quantum_outputs
set explicitly

    def forward(self, x_quantum, x_classical):
        q_out = self.qlayer(x_quantum)
        c_out = torch.relu(self.classical(x_classical))
        combined = torch.cat([q_out, c_out], dim=1)
        return self.head(combined)
```

> **Note (Fault #8 fix):** `qml.qnn.TorchLayer` does NOT have an `out_features` attribute. The quantum output dimension must be set explicitly as a constructor parameter (e.g., `n_quantum_outputs = n_qubits`).

Gotchas & Constraints

- PennyLane's parameter-shift gradients are significantly slower than PyTorch native autograd. Expect 10-100x slowdown vs. classical networks.
- Use `torch.no_grad()` during evaluation to avoid unnecessary gradient computation.
- Set `torch.manual_seed(seed)` AND `np.random.seed(seed)` for reproducibility.
- CPU-only training is fine for VQC (quantum simulation is the bottleneck, not tensor operations).

Tool: Scikit-learn

Version to use: `1.6.1` (current stable as of 2026) **Install command:** `pip install scikit-learn==1.6.1` **Used in:** Phase 3 (baselines), Phase 4 (hybrid SVC), Phase 6 (evaluation) **Purpose in QGenomix:** Train classical baselines (LR, RF, SVM); provide `SVC(kernel='precomputed')` for quantum kernel classification; cross-validation; metrics computation.

How to Use It

```
from sklearn.svm import SVC
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score

# Quantum Kernel SVC
svc = SVC(kernel='precomputed', probability=True, class_weight='balanced')
```

```
svc.fit(K_train, y_train)  # K_train = quantum kernel matrix (N_train × N_train)
y_prob = svc.predict_proba(K_test)[:, 1]  # K_test = kernel matrix (N_test × N_train)
auc = roc_auc_score(y_test, y_prob)
```

Gotchas & Constraints

- `SVC(kernel='precomputed')` requires a square kernel matrix for training and a `(N_test, N_train)` matrix for prediction. **Always use `np.ix_` for subsetting:** `K_train = K[np.ix_(train_idx, train_idx)]`.
- `class_weight='balanced'` is essential for imbalanced data.
- `probability=True` adds computational overhead to SVC (Platt scaling) but is needed for AUC computation.
- Random state must be set in both the model AND cross-validation for reproducibility.

Tool: XGBoost

Version to use: `2.1.3` **Install command:** `pip install xgboost==2.1.3` **Used in:** Phase 3 (baseline), Phase 4 (hybrid ensemble component) **Purpose in QGenomix:** High-performance gradient boosted trees for classical baseline and as a component of the hybrid stacking ensemble; provides tree-based SHAP via TreeExplainer.

How to Use It

```
import xgboost as xgb

# Compute class imbalance ratio explicitly
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()

model = xgb.XGBClassifier(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.01,
    scale_pos_weight=scale_pos_weight,
    eval_metric='auc',
    early_stopping_rounds=50,
    random_state=42
)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=False)
```

> **Note (Fault #9 fix):** `scale_pos_weight='auto'` is NOT a valid value. Always compute it as `(y==0).sum() / (y==1).sum()` explicitly.

Gotchas & Constraints

- `scale_pos_weight` must be recomputed per fold if class ratio changes.
- `eval_metric='auc'` requires binary labels.
- For very high-dimensional input (>10,000 features), XGBoost can be slow. Use `tree_method='hist'` for speed.

- `early_stopping_rounds` requires `eval_set` to be specified in `fit()`.

Tool: SHAP

Version to use: `0.46.0` **Install command:** `pip install shap==0.46.0` **Used in:** Phase 5 (explainability + adaptive loop) **Purpose in QGenomix:** Compute Shapley values to rank SNP pair importance via the XGBoost sub-model; enable the adaptive feedback loop.

How to Use It

```
import shap

# Use TreeExplainer on XGBoost sub-model ONLY (fast, exact)
explainer = shap.TreeExplainer(hybrid_model.xgb_model)
shap_values = explainer(X_train)

# DO NOT use KernelExplainer on the full hybrid model —
# it would trigger O(N²) kernel computation per SHAP sample
```

Example Code Snippet

```
import shap
import numpy as np

# Rank SNP pairs by combined SHAP importance
shap_vals = explainer(X)
mean_abs_shap = np.abs(shap_vals.values).mean(axis=0)

# Map to pair importance
pair_importance = {}
for idx_a, idx_b in epistasis_pairs:
    pair_importance[(idx_a, idx_b)] = mean_abs_shap[idx_a] +
mean_abs_shap[idx_b]

ranked_pairs = sorted(pair_importance.items(), key=lambda x: x[1], reverse=True)
top_k_pairs = [pair for pair, score in ranked_pairs[:100]]
```

Gotchas & Constraints

- **DO NOT use `shap.KernelExplainer` on the hybrid model** — it triggers O(N²) kernel computation per SHAP sample, making it intractable. Always use `TreeExplainer` on the XGBoost component only.
- For > 1000 features, SHAP computation can take hours. Use a subset or limit to top features from initial importance ranking.
- `TreeExplainer` with `feature_perturbation="interventional"` avoids issues with correlated features (important for SNPs in LD).
- SHAP interaction values (`explainer.shap_interaction_values()`) are only available for TreeExplainer and are O(N × P²) — very expensive.
- **API (Fault #15 fix):** `shap.waterfall_plot` was deprecated in SHAP 0.40+. Use `shap.plots.waterfall(shap.Explanation(...))` instead.

Tool: Pandas

Version to use: `2.2.3` **Install command:** `pip install pandas==2.2.3` **Used in:** All phases (data processing, results management) **Purpose in QGenomix:** Load and manipulate genomic metadata, GWAS summary stats, phenotype files, experiment results, and plink2 `--export A` output (`.raw` files).

Gotchas & Constraints

- Large genomic files (>1GB): use `chunksize` parameter in `read_csv` for streaming.
- SNP IDs (rsIDs) should be treated as strings, not integers.
- Memory: for 500K SNPs × 2504 samples as float32, the matrix is ~5GB in Pandas. Use NumPy directly for large matrices.

Tool: NumPy

Version to use: `1.26.4` **Install command:** `pip install numpy==1.26.4` **Used in:** All phases (core numerical computation) **Purpose in QGenomix:** Store and manipulate SNP matrices, compute PRS, handle array operations for quantum feature vectors.

Gotchas & Constraints

- Use `dtype=float32` for large matrices to save memory (vs. default float64).
- `np.save`/`np.load` is the fastest serialization for NumPy arrays.
- Pin NumPy <2.0 to avoid breaking changes with PennyLane.

Tool: Matplotlib / Seaborn

Version to use: Matplotlib `3.9.3`, Seaborn `0.13.2` **Install command:** `pip install matplotlib==3.9.3 seaborn==0.13.2` **Used in:** Phase 3 (baseline plots), Phase 5 (SHAP plots), Phase 6 (final report figures)

Tool: plink2

Version to use: `2.00a6` (or latest alpha) **Install command:** Download binary from https://www.cog-genomics.org/plink/2.0/ — no pip install **Used in:** Phase 1 (data QC)

How to Use It

```
# No merge step needed — 1000 Genomes Phase 3 is available as a single genome-wide file
```

```
# QC filtering
plink2 --pfile data/raw/1kg_phase3/all_phase3 \
       --maf 0.01 \
       --hwe 1e-6 \
       --geno 0.05 \
       --mind 0.1 \
       --max-alleles 2 \
       --snps-only just-acgt \
       --make-pgen \
       --out data/interim/qc_filtered

# LD pruning
plink2 --pfile data/interim/qc_filtered \
       --indep-pairwise 50 5 0.2 \
       --out data/interim/pruning

plink2 --pfile data/interim/qc_filtered \
       --extract data/interim/pruning.prune.in \
       --make-pgen \
       --out data/interim/qc_pruned

# Export to additive dosage format (for Python — read .raw with Pandas)
plink2 --pfile data/interim/qc_pruned \
       --export A \
       --out data/processed/snp_dosage
```

Gotchas & Constraints

- plink2 is a binary — must be in PATH or referenced by full path.
- Memory: may require 16+ GB RAM for full dataset. Use `--memory` flag to set limit.
- `--export A` produces a `.raw` file (tab-separated) that Pandas can read directly — **no `pandas_plink` package needed**.
- plink2 is still in "alpha" but is stable for standard operations.

Tool: Git / GitHub Actions

Version to use: Git `2.40+`, GitHub Actions (latest runners) **Used in:** Phase 7 (CI/CD), all phases (version control)

How to Use It

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.10'
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: make lint
      - run: make test
```

Tool: SLURM (HPC Job Scripts)

Version to use: Depends on university cluster

How to Use It

```
#!/bin/bash
#SBATCH --job-name=qgenomix-vqc
#SBATCH --output=results/logs/vqc_%j.out
#SBATCH --error=results/logs/vqc_%j.err
#SBATCH --time=12:00:00
#SBATCH --mem=32G
#SBATCH --cpus-per-task=8
#SBATCH --partition=compute

module load python/3.10
source venv/bin/activate
python scripts/run_pipeline.py --stage vqc --config configs/vqc_8qubit.yaml --
seed 42
```

Section F — Standard Production Procedures

F.1 — Git & Version Control Protocol

Branch Naming Convention

```
main                          # Production-ready code
develop                       # Integration branch
feature/<chapter>-<name>      # New feature (e.g., feature/ch2-vqc-encoder)
fix/<issue>-<name>            # Bug fix (e.g., fix/shap-nan-values)
docs/<section>                # Documentation (e.g., docs/setup-guide)
experiment/<name>             # Experimental branches (e.g., experiment/10qubit-
vqc)
```

Commit Message Format (Conventional Commits)

```
<type>(<scope>): <subject>

Types: feat, fix, docs, test, refactor, perf, chore
Scope: data, quantum, classical, hybrid, explain, benchmark, utils, ci
```

Release Tagging

Tags are created by **Mem2** (as CI/CD owner) at the end of each phase:

```
v0.1.0 — Data pipeline complete (end of Week 2)
v0.2.0 — VQC + quantum kernel complete (end of Week 5)
v0.3.0 — Classical baselines complete (end of Week 7)
v0.4.0 — Hybrid model complete (end of Week 11)
v0.5.0 — Explainability + adaptive loop complete (end of Week 13)
v1.0.0 — Final submission (end of Week 16)
```

Command: `git tag -a v0.1.0 -m "Data pipeline complete" && git push origin v0.1.0`

Large Data & Model Files

- **Rule:** NEVER commit data files or model weights to Git.
- `.gitignore` covers: `data/raw/`, `data/interim/`, `data/processed/`, `results/models/`, `*.pgen`, `*.pvar`, `*.npy`, `*.pt`, `*.pkl`
- Use `.gitkeep` files to preserve empty directory structure.
- **Model hosting (Fault #7 fix):** Trained model files (`vqc_best.pt`, `hybrid_best.pkl`, `xgb_baseline.pkl`) are hosted on **Google Drive or Zenodo** (not committed to Git). A `scripts/download_models.sh` script downloads them. The `results/README.md` documents the external URLs.
- Share data via Google Drive or university file share.

F.3 — Dependency Management

Requirements Files

`requirements.txt` (production — pinned versions):

```
pennylane==0.40.0
pennylane-lightning==0.40.0
torch==2.5.1
scikit-learn==1.6.1
xgboost==2.1.3
shap==0.46.0
pandas==2.2.3
numpy==1.26.4
matplotlib==3.9.3
seaborn==0.13.2
pyyaml==6.0.2
joblib==1.4.2
scipy==1.14.1
statsmodels==0.14.4
```

> **Fault #3/#31/#32 fixes:** Added `statsmodels`, `pennylane-lightning`. Removed `pandas_plink` (replaced by `plink2 --export A` + Pandas).

`requirements-dev.txt` (development tools):

```
pytest==8.3.4
pytest-cov==6.0.0
black==24.10.0
flake8==7.1.1
mypy==1.13.0
isort==5.13.2
pre-commit==4.0.1
jupyterlab==4.3.4
nbstripout==0.7.1
mkdocs==1.6.1
mkdocs-material==9.5.44
```

```
mkdocstrings[python]==0.27.0
```

Virtual Environment Setup

```
# Option A: venv
python -m venv venv
source venv/bin/activate       # Linux/Mac
.\venv\Scripts\activate         # Windows
pip install -r requirements.txt
pip install -r requirements-dev.txt

# Option B: conda
conda create -n qgenomix python=3.10
conda activate qgenomix
pip install -r requirements.txt
pip install -r requirements-dev.txt
```

F.5 — Testing Protocol

How to Test Quantum Circuits

```
import pennylane as qml
import numpy as np

dev = qml.device("default.qubit", wires=2)

@qml.qnode(dev)
def test_circuit(x):
    qml.RY(x * np.pi / 2, wires=0)
    return qml.state()

# Test: dosage 0 should give |0⟩
state = test_circuit(0.0)
assert np.allclose(state, [1, 0, 0, 0], atol=1e-6)  # |00⟩

# Test: dosage 2 should give |1⟩ (on wire 0)
# State vector indices for 2 wires: 0→|00⟩, 1→|01⟩, 2→|10⟩, 3→|11⟩
state = test_circuit(2.0)
assert np.allclose(np.abs(state[2])**2, 1.0, atol=1e-6)  # |10⟩ — wire 0 flipped
```

> **Note (Fault #17 fix):** The device has `wires=2` (indices 0 and 1). The `@qml.qnode(dev)` correctly uses wire 0. State index 2 corresponds to |10⟩ (wire 0 = 1, wire 1 = 0).

F.7 — Build & Run Scripts

Makefile Targets

```
.PHONY: install data qc train evaluate test lint clean clean-data docs

install:
    pip install -r requirements.txt
    pip install -r requirements-dev.txt
    pre-commit install
```

```
data:
    bash scripts/download_data.sh

qc:
    bash scripts/run_qc.sh

train:
    python scripts/run_pipeline.py --stage all --config configs/default.yaml

train-vqc:
    python scripts/run_pipeline.py --stage vqc --config configs/vqc_8qubit.yaml

train-classical:
    python scripts/run_pipeline.py --stage classical --config
configs/classical_baselines.yaml

evaluate:
    python scripts/run_pipeline.py --stage benchmark --config
configs/default.yaml

test:
    pytest tests/ -v --cov=src --cov-report=term-missing

test-fast:
    pytest tests/ -v -m "not slow"

lint:
    black --check src/ tests/ scripts/
    flake8 src/ tests/ scripts/
    mypy src/

format:
    black src/ tests/ scripts/
    isort src/ tests/ scripts/

clean:
    find . -name "*.pyc" -delete
    find . -name "__pycache__" -delete
    rm -rf site/

clean-data:
    @echo "WARNING: This will delete ALL processed data and model files."
    @echo "Press Ctrl+C within 5 seconds to cancel."
    @sleep 5
    rm -rf data/interim/* data/processed/* results/models/* results/logs/*

docs:
    mkdocs serve -f docs/mkdocs.yml

docs-build:
    mkdocs build -f docs/mkdocs.yml -d site/
```

> **Fault #16 fix:** Split `clean` into `clean` (safe — only Python cache + build artifacts) and `clean-data` (dangerous — deletes processed data with a 5-second confirmation delay).

F.9 — Final Results Scene

What "Done" Looks Like

Deliverable	Completion Criteria
Software Pipeline	`make train && make evaluate` runs end-to-end without error
Technical Documentation	`mkdocs serve` shows all pages; all functions documented
Demo Notebooks	All 6 notebooks run end-to-end; outputs reproducible
Benchmarked Study	`results/tables/full_cv_results.csv` has 8 models × 5 folds + 1 adaptive model on fixed test split
Academic Paper	8-12 pages (IEEE BIBM format), all figures, ≥20 references
Repository	CI passes, README complete, .gitignore correct

Required Files in `results/` at Completion

```
results/
├── figures/
│   ├── roc_comparison.png                # ROC curves for all models
│   ├── baseline_comparison_val.png       # Bar chart of baseline models
(validation)
│   ├── adaptive_loop_convergence.png     # AUC vs iteration
│   ├── shap_summary.png                  # SHAP beeswarm plot
│   ├── snp_pair_importance.png           # Top SNP pairs bar chart
│   ├── kernel_matrix_heatmap.png         # Quantum kernel visualization
│   ├── edr_curve.png                     # EDR@K for top models
│   └── qubit_scaling.png                 # AUC + runtime vs n_qubits
├── tables/
│   ├── baseline_val_results.csv          # Validation eval for baselines
│   ├── full_cv_results.csv               # 5-fold CV for all models
│   ├── statistical_tests.csv             # DeLong + McNemar results
│   ├── full_comparison.tex               # LaTeX table
│   └── snp_pair_ranking.csv              # Final ranked SNP pairs
├── models/                               # NOT committed — hosted on Google
Drive/Zenodo
│   ├── .gitkeep                          # Directory structure preserved
│   └── MODELS_URL.txt                    # URL to download pre-trained models
└── logs/
    ├── vqc_training.csv                  # VQC training log
    ├── adaptive_loop_log.csv             # Adaptive loop iterations
    └── experiment_configs/               # All experiment configs
```

> **Fault #7/#40 fix:** Model files are NOT committed to Git. They are hosted externally. `results/models/MODELS_URL.txt` contains the download URL and is committed. The repository readiness checklist verifies that `results/figures/` and `results/tables/` are populated (these ARE committed), and that `results/models/MODELS_URL.txt` exists.

Repository Readiness Checklist

- ☐ `git status` is clean (no uncommitted changes)
- ☐ All tests pass: `make test`
- ☐ Linting passes: `make lint`
- ☐ README renders correctly on GitHub
- ☐ All notebooks execute without error
- ☐ `data/` has no committed data files (only `.gitkeep`)
- ☐ `results/figures/` and `results/tables/` are populated (committed)
- ☐ `results/models/MODELS_URL.txt` exists with valid download URL
- ☐ Documentation builds: `make docs-build`
- ☐ CI/CD pipeline passes
- ☐ Paper draft is complete and references repo

Section G — Risk Register & Mitigation Plan

#	Risk	Likelihood	Impact	Mitigation Strategy	Contingency Plan
1	**Quantum simulator memory ceiling hit at ≥12 qubits**	M	M	Design VQC for 8 qubits (safe); test 10-qubit variant via qubit-scaling experiment. `default.qubit` at 12 qubits uses ~65 KB per state vector, ~163 MB for 2504 batched samples — manageable. Risk is at 16+ qubits.	Cap at 8 qubits for all experiments. Use `lightning.qubit` for 2-3x memory efficiency. Report qubit scaling analysis as a result (show runtime vs. n_qubits).
2	**1000 Genomes does not have T2D phenotype labels**	**H**	**H**	**CONFIRMED:** 1000 Genomes has NO disease phenotypes. **Primary**	If simulated phenotypes are deemed insufficient by reviewer: (a) Frame as

#	Risk	Likelihood	Impact	Mitigation Strategy	Contingency Plan
				mitigation: Use **injected epistatic phenotypes** — simulate labels from known pair interactions (`liability = Σ(β_ij × SNP_i × SNP_j) + ε`) with NO linear PRS contribution. This avoids circular validation and directly supports the epistasis detection claim. **Secondary:** Use PennyLane genomic dataset (805 SNPs, pre-labeled) as proof-of-concept.	"method validation on synthetic epistatic data with known ground truth", (b) Use UK Biobank data if access can be obtained, (c) The known ground-truth pairs STRENGTHEN the EDR metric.
3	**VQC training fails to converge (barren plateau)**	M	H	Mitigations: (a) Use shallow circuits (≤6 layers), (b) Use local cost functions (measure individual qubits), (c) Initialize parameters near zero, (d) Use Adam optimizer with warmup schedule, (e)	Reduce to 4-qubit VQC, use data re-uploading strategy, report barren plateau analysis.

#	Risk	Likelihood	Impact	Mitigation Strategy	Contingency Plan
				Monitor gradient norms — if < 1e-6, reduce circuit depth.	
4	**SHAP computation intractable on full feature set**	M	M	Use `TreeExplainer` (fast, exact) on XGBoost sub-model ONLY. Do NOT use `KernelExplainer` on the hybrid model — it triggers $O(N^2)$ kernel computation per SHAP sample. Document this limitation explicitly.	Use permutation importance instead of SHAP as ranking signal.
5	**Team member unavailability / workload imbalance**	M	M	Gantt chart ensures parallel work streams. Workload redistributed in weeks 14-16 (CI/CD → Mem2, README → Mem3). Critical path is Mem2's quantum work.	If Mem2 unavailable: Mem3 takes over VQC training. If Mem1 unavailable: data pipeline is scripted. If Mem3 unavailable: benchmarking is automated.
6	**HPC cluster access denied or quota exceeded**	L	M	Request HPC allocation early (Week 1). Estimate total compute: ~500 CPU-hours for VQC training, ~200 CPU-hours for kernel, ~100 CPU-hours for	Use Google Colab Pro ($10/month). VQC training on 8 qubits can run on a modern laptop in ~12 hours.

#	Risk	Likelihood	Impact	Mitigation Strategy	Contingency Plan
				baselines.	
7	**Quantum kernel matrix computation becomes O(n^2) bottleneck**	H	M	For N=2504: ~3.1M kernel evaluations. ~8.6 hours on `default.qubit`, ~3 hours on `lightning.qubit`. Mitigations: parallelize across CPU cores, cache entries, Nyström approximation. **During adaptive loop: skip kernel recomputation entirely** — use VQC features + XGBoost only.	Nyström approximation (M=200 landmarks), random Fourier features, or drop kernel SVC from ensemble.
8	**No statistically significant improvement over classical baseline**	M	H	Use epistatic phenotype simulation where interaction effects are injected — ensures quantum model has a theoretical advantage on EDR. Report EDR as a novel metric where quantum may excel even if AUC is similar.	Frame as "characterizing conditions under which quantum advantage emerges". Analyze WHERE quantum outperforms (specific subgroups).

Section H — Glossary

Term	Definition
SNP (Single Nucleotide Polymorphism)	A variation in a single DNA base pair at a specific position in the genome. Encoded as 0 (homozygous reference), 1 (heterozygous), or 2 (homozygous alternative).
Epistasis	A phenomenon where the combined effect of two or more SNPs on a phenotype differs from the sum of their individual effects. Non-linear gene-gene interactions.
PRS (Polygenic Risk Score)	A linear weighted sum of allele dosages: PRS = Σ(β_i × dosage_i). Cannot capture epistasis.
VQC (Variational Quantum Circuit)	A parameterized quantum circuit optimized via classical optimization. In QGenomix, encodes SNP pairs into entangled quantum states.
CZ Gate (Controlled-Z Gate)	A two-qubit gate applying a phase flip when the control qubit is $\|1\rangle$. Creates entanglement between qubits representing different SNPs.
Parameter-Shift Rule	Method for computing exact gradients: $\partial f/\partial\theta = [f(\theta + \pi/2) - f(\theta - \pi/2)] / 2$.
Quantum Kernel	$K(x, x') = \|\langle\varphi(x')\|\varphi(x)\rangle\|^2$ — computed via quantum circuit fidelity in exponentially large Hilbert space.
SHAP (SHapley Additive exPlanations)	Game-theoretic feature importance. In QGenomix, computed on XGBoost sub-model only.
GWAS (Genome-Wide Association Study)	Genome-wide scan for SNP-trait associations. Produces p-values and effect sizes.
AUC-ROC	Area Under ROC Curve. Primary evaluation metric. AUC=1.0 is perfect, 0.5 is random.
Entanglement	Quantum correlation where qubits cannot be described independently. Enables modeling epistasis.
Hilbert Space	2^n-dimensional vector space for n qubits. Basis for potential quantum advantage.
Qubit	Fundamental quantum information unit. Superposition: $\|\psi\rangle = \alpha\|0\rangle + \beta\|1\rangle$.
Angle Encoding	Map classical data to rotation angles. QGenomix: dosage {0,1,2} → Ry angles {0, π/2, π}.
Barren Plateau	Exponentially small gradients in deep variational circuits. Quantum analog of vanishing gradients.
MAF (Minor Allele Frequency)	Frequency of the less common allele. SNPs with

Term	Definition
	MAF < 1% are filtered out.
HWE (Hardy-Weinberg Equilibrium)	Expected genotype frequencies in stable populations. Deviations may indicate genotyping errors.
LD (Linkage Disequilibrium)	Non-random association of nearby alleles. LD pruning removes redundant SNPs.
Nyström Approximation	Approximate N×N kernel matrix using M<<N landmark points. Reduces $O(N^2)$ to O(MN).
Stacking Ensemble	Meta-learning: Level 1 model predictions become Level 2 features. QGenomix uses quantum kernel SVC + XGBoost → logistic regression.
Cross-Validation	K-fold model evaluation. QGenomix: stratified 5-fold CV with kernel matrix subsetting.
DeLong Test	Statistical test comparing two correlated AUC-ROC values.
McNemar Test	Statistical test comparing two classifiers' disagreements on the same test data.
Linear-Ladder Connectivity	CZ gate pattern alternating between even-pair (0-1, 2-3, 4-5, 6-7) and odd-pair (1-2, 3-4, 5-6) entanglement layers. Ensures all adjacent qubits are entangled within 2 layers.

Next Action

The single most important thing the team should do in the first 24 hours:

> **Mem1:** Run `scripts/download_data.sh` to begin downloading the 1000 Genomes Phase 3 PLINK files from https://www.cog-genomics.org/plink/2.0/resources#1kg_phase3. This download can take 4-8 hours depending on bandwidth and should start immediately. While it downloads:
>
> 1. **All team members:** Set up the development environment (clone repo, create venv, install dependencies from `requirements.txt`).
> 2. **Mem1:** Begin downloading T2D GWAS summary statistics from the DIAGRAM consortium (for PRS, NOT for phenotype simulation).
> 3. **Mem2:** Complete the PennyLane VQC tutorial at https://pennylane.ai/qml/demos/tutorial_variational_classifier.html and design the 8-qubit circuit architecture on paper (CZ connectivity pattern, layer structure).

4. **Mem3:** Write `src/classical/metrics.py` and `src/utils/reproducibility.py` — these have zero dependencies and can be completed immediately.

By end of Day 1: Data download is running, dev environments are set up, circuit design is sketched on paper, metrics module is testable.

End of QGenomix Project Plan. Document version 2.0 (Fault-corrected). Generated 2026-05-25.